

BUS TICKET RESERVATION SYSTEM

A Project Report Submitted in partial fulfilment of the requirements for the award of
the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

N.V.V.G.MANIKANTA

22221A0582

K.HEMANTH

22221A0553

K.A.G.TEJA

22221A0551

Under the Esteemed Guidance of

Mr.M.V.K.SUBASH

M. Tech.,(Ph.D)

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAMVENKATA CHALAMAYYA ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Permanently Affiliated to J. N.T.U.K, Kakinada)

(Accredited by N.B.A & NAAC with 'A' Grade)

ODALAREVU – 533210

2022-2026

BONAMVENKATA CHALAMAYYA ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Permanently Affiliated to J. N.T.U.K, Kakinada)

(Accredited by N.B.A & NAAC with 'A' Grade)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project work entitled **“BUS TICKET RESERVATION SYSTEM”** is being submitted for the partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, at **BVC Engineering College, Odalarevu**, is a bonafide work done by **N.V.V.G.MANIKANTA (22221A0582)**, **K.HEMANTH (22221A0553)**, **K.A.G.TEJA(22221A0551)** under my guidance during the academic year 2025-2026 and it has been found suitable for acceptance according to the requirement of the University.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree.

Project Guide

Mr.M.V.K.SUBASH

M.Tech(Ph.D)

Assistant Professor

Head of the Department

Dr. MEERASHARIF SHEIK

M.Tech,Ph.D

Professor & HOD

External Examiner

ACKNOWLEDGEMENT

We profoundly grateful to express our deep sense of gratitude and respect towards our guide **Mr.M.V.K.SUBASH**, M.Tech,(Ph.D), Assistant Professor of CSE Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his excellent guidance right from selection of the project and his valuable suggestions throughout the project work.

His constant encouragement and support has been the cause of our success, in completing this thesis in the college.

We also grateful to the Project Co-Ordinator **Mr.M.V.K.SUBASH**, M.Tech.,(Ph.D) Assistant Professor of CSE Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for her excellent guidance right from selection of the project and her valuable suggestions throughout the project work.

We really thankful to **Dr. MEERASHARIF SHEIK**, M.Tech,Ph.D, Professor and Head of the Department of Computer Science and Engineering for providing the laboratory facilities to the fullest extent as and when required and also for giving us the opportunity to carry out the project work in the college.

We also thankful to **Dr. MAHESWAR DUTTA**, M.E,Ph.D, Principal, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his support during and till the completion of the project. And also thankful to our **MANAGEMENT**, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for their support during and till the completion of the project.

We are thankful to all the **TEACHING AND NON-TEACHING STAFF** of Computer Science and Engineering Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, and **FRIENDS** for their direct and indirect help provided to us in completing the project.

Last but not least, we own all our success to our **FAMILY MEMBERS, CLASSMATES AND TEACHERS** from our child hood, whose vision, love and inspiration made us to reach out for these glories.

PROJECT ASSOCIATES

N.V.V.G.MANIKANTA

22221A0582

K.HEMANTH

22221A0553

K.A.TEJA

22221A0551

ABSTRACT

The Bus Ticket Management System also includes essential administrative functionalities to manage bus journey records efficiently. Administrators can add, update, and remove bus details such as route information, timings, seat availability, and ticket pricing. This centralized management ensures accurate data handling and helps maintain consistency across all booking operations. By storing journey and ticket information in a structured database, the system ensures data persistence, reliability, and easy retrieval of records whenever required.

From a technical perspective, this project helps in understanding the complete backend workflow of a real-world web application. It demonstrates how RESTful APIs handle HTTP requests and responses, how Spring Boot simplifies application configuration, and how MySQL manages relational data. The project follows a layered architecture separating controller, service, and repository components, making the code clean, modular, and maintainable. Overall, this system serves as a strong foundation for learning backend development concepts and can be further enhanced with features such as authentication, payment integration, and real-time seat tracking.

LIST OF CONTENTS

CONTENTS	PAGE NO
CHAPTER-1	8
INTRODUCTION	8
CHAPTER-2	9
LITERATURE SURVEY	9
CHAPTER-3	10
SYSTEM ANALYSIS	10
3.1 Existing System	10
3.1.1 Drawbacks	11
3.2 Proposed System	11
3.2.1 Advantages	11
CHAPTER-4	13
REQUIREMENT ANALYSIS	13
4.1 Hardware Requirements	13
4.2 Software Requirements	13
CHAPTER-5	14
SOFTWARE DESIGN	14
5.1 System Architecture	14
5.2 UML Diagrams	14
5.2.1 Class Diagram	14
5.2.3 Sequence Diagram	16
CHAPTER-6	17
SYSTEM ENVIRONMENT	17
6.1 What is Java?	17
6.2 Advantages of Java over Other Languages	18
6.3 Disadvantages of Java	19
6.4 History of Java	19
6.5 What is Spring Boot?	20
6.6 What is REST API	20
6.7 Database Environment – MySQL	21
6.8 Development Tools Used	21
CHAPTER-7	22
TECHNOLOGIES USED	22
7.1 User Module	22

.....	22
7.2 Admin Module	23
7.3 Booking Module.....	23
CHAPTER-8	25
TESTING	25
8.1 System Testing	25
8.2 Types of Testing	25
8.2.1 Unit Testing	25
8.2.2 Integrated Testing	26
8.2.4 Functional test	27
8.2.4 White Box Testing	27
8.2.5 Black Box Testing.....	28
8.2.6 White Box Testing	28
CHAPTER – 9.....	31
IMPLEMENTATION.....	31
9.1 Methodology	31
9.2 Source Code	32
CHAPTER –10.....	40
IMPLEMENTATION,RESULTSAND SCREENSHOTS	40
10.1 Introduction	40
10.2 System Modules	40
10.2.1 User Login Module	40
10.2.2 Bus Search Module	41
10.2.3 Ticket Booking Module	41
10.2.4 Admin Module.....	42
10.3 Running the Code	43
10.3.1 Install Java JDK (version 17 or above).....	43
CHAPTER – 11.....	46
CONCLUSION.....	46
CHAPTER – 12.....	47
REFERENCES	47

LIST OF FIGURES

FIGURES	PAGENO
Fig 5.1 System Architecture	14
Fig 5.2 Class Diagram	15
Fig 5.3 Use Case Diagram.....	15
Fig 5.4 Sequence Diagram	16
Fig 7.1 Show User Interface for Searching Buses.....	22
Fig 7.2 Display List of available buses.....	23
Fig 7.3 Illustrate the Ticket Booking Module.....	24
Fig 7.4 Show Ticket Confirmation Screen.....	24
Fig 8.1 Output of UnitTesting.....	26
Fig 8.2 Represents the Integration Testing Flow.....	26
Fig 10.1 User Login Module	40
Fig 10.2 Bus Search Module.....	41
Fig 10.3 Booking Ticket Module	42
Fig 10.4 Admin Login Module	42
Fig 10.5 Installation of Java Development Kit 17	43
Fig 10.6 Installation of My SQL Server	43
Fig 10.7 Open Project in Visual Studio Code.....	44
Fig 10.8 Update DataBase Configuration in application.properties.....	44
Fig10.9 Start application by running Main Application	45

CHAPTER-1

INTRODUCTION

The **Bus Ticket Reservation System** is a web-based application designed to automate and simplify the process of booking and managing bus tickets. Traditionally, bus ticket booking involved manual processes such as physical counters, paperwork, and human intervention, which were time-consuming and prone to errors. With the rapid growth of digital technologies, there is a strong need for an automated system that can efficiently manage bus journeys and ticket reservations.

This project provides an online platform where users can view available bus journeys based on source, destination, date, and time. Users can perform essential operations such as ticket booking and cancellation in a simple and efficient manner. The system reduces manual effort, minimizes errors, and improves overall efficiency by maintaining all journey and ticket information in a centralized database.

The application is developed using Java, Spring Boot, RESTful APIs, and MySQL, which together provide a robust and scalable backend architecture. Spring Boot simplifies application development by reducing configuration complexity, while REST APIs enable smooth communication between the client and server. MySQL is used to store and manage bus, journey, and ticket data securely.

The primary objective of this project is to demonstrate core backend development concepts such as API design, database interaction, layered architecture, and CRUD operations. This system is suitable for academic purposes and beginner-level projects, and it can be further enhanced by adding features like user authentication, online payment integration, seat availability tracking, and admin dashboards.

CHAPTER-2

LITERATURE SURVEY

A literature survey provides an overview of existing systems and technologies related to the **Bus Ticket Reservation System** and helps in understanding how similar problems have been addressed previously. Several online ticket booking platforms such as railway reservation systems, airline booking systems, and private bus booking applications have influenced the design of this project. These systems demonstrate the importance of automation, centralized data storage, and user-friendly interfaces in managing transportation services efficiently.

Earlier bus ticket booking systems were mostly **manual or semi-automated**, where passengers had to visit booking counters to check seat availability and purchase tickets. These systems depended heavily on human operators, leading to long waiting times, data inconsistency, and higher chances of errors. Maintaining records in paper form also made it difficult to retrieve historical booking information and generate reports.

With the advancement of web technologies, **online reservation systems** became popular. Many modern systems use web-based applications and APIs to allow users to search journeys, book tickets, and cancel reservations from anywhere. Technologies such as Java-based frameworks, RESTful web services, and relational databases have proven to be reliable and scalable for such applications. These systems ensure faster response times, better data management, and improved user experience.

Based on the study of existing ticket reservation platforms, it was observed that a **simple, lightweight backend system** is ideal for academic and beginner projects. Therefore, this project focuses on implementing core functionalities like bus search, ticket booking, and cancellation using **Spring Boot and REST APIs**, without unnecessary complexity. The literature survey helped in identifying best practices such as layered architecture, proper API design, and secure database interaction, which are applied in this project.

CHAPTER-3

SYSTEM ANALYSIS

System analysis involves studying the existing system, identifying its limitations, and proposing an improved solution. This chapter explains the current bus ticket booking process, its drawbacks, and how the proposed system overcomes those limitations using modern technologies.

3.1 Existing System

1. Manual Bus Ticket Booking Systems

Earlier bus ticket booking systems were completely manual. Passengers had to visit bus stations or ticket counters to enquire about bus schedules, seat availability, and fares. Tickets were issued by staff using handwritten records or printed slips. These systems required significant human effort, were time-consuming, and were prone to errors such as double booking and loss of records.

2. Semi-Computerized Booking Systems

To reduce manual work, some bus operators introduced basic computer-based systems where booking details were stored digitally. However, these systems were limited to local machines and lacked network connectivity. Data sharing between counters was difficult, and real-time seat availability was not always accurate. Cancellation and report generation were also inefficient.

3. Web-Based Reservation Systems

With the growth of the internet, web-based bus ticket reservation systems were introduced. These systems allowed users to search buses, view schedules, and book tickets online. While they improved accessibility and reduced manual effort, many early systems had poor scalability, complex user interfaces, and limited backend flexibility.

4. API-Based Reservation Systems

Modern reservation systems use RESTful APIs to handle communication between clients and servers. These systems separate frontend and backend logic, improving maintainability and scalability. APIs allow operations such as searching journeys, booking tickets, and cancelling reservations to be handled efficiently. However, many systems are complex and difficult for beginners to understand.

5. Spring Boot-Based Backend Systems

Recent applications use Java Spring Boot for backend development due to its simplicity, embedded server support, and reduced configuration. Spring Boot-based systems provide better performance, structured architecture, and easy database integration using ORM tools. These systems are reliable and suitable for real-world and academic projects.

6. Limitations of Existing Systems

Despite advancements, existing bus ticket reservation systems still face challenges:

- High development complexity in large-scale systems
- Dependency on heavy frontend frameworks
- Limited flexibility for academic or beginner projects
- Difficulty in understanding system architecture for learning purposes

3.1.1 Drawbacks

1. Time Consuming Process

Booking and cancellation require physical presence, leading to long waiting times.

2. High Human Dependency

Manual data entry increases the possibility of errors such as incorrect bookings and double reservations.

3. Poor Data Management

Records stored on paper or local systems are difficult to maintain and retrieve.

4. Lack of Centralized Database

Data is not synchronized across multiple counters or systems.

5. Limited Accessibility

Users cannot access booking services remotely or at any time.

6. Scalability Issues

The system cannot efficiently handle large numbers of passengers and bookings.

3.2 Proposed System

The proposed Bus Ticket Reservation System is a web-based application designed to automate the entire ticket booking process. It allows users to search for available buses based on source, destination, date, and time. Users can easily book and cancel tickets through RESTful APIs without visiting physical counters.

The system is developed using Java, Spring Boot, REST APIs, and MySQL, ensuring secure data storage and efficient processing. All bus journey and ticket information is stored in a centralized database, providing fast access and better data consistency. The system follows a layered architecture that improves maintainability and scalability.

3.2.1 Advantages:

1. Fully Automated System

Eliminates manual intervention in booking and cancellation processes.

2.Reduced Human Errors

Automated validation and database handling minimize mistakes.

3.Centralized Database Management

Ensures secure storage and easy retrieval of records.

4.Time Efficient

Faster booking and cancellation operations.

5.User-Friendly Access

Can be accessed from anywhere using web services.

6.Scalable and Maintainable

Easy to enhance with additional features in the future.

7.Academic and Beginner Friendly

Simple design helps in understanding backend development concepts.

CHAPTER-4

REQUIREMENT ANALYSIS

4.1 Hardware Requirements

- Processor: Intel i5 or Above
- Ram: Minimum 4GB
- Secondary storage: Minimum 250 GB
- Internet Connection: Required
- Input Devices: Keyboard, Mouse
- Output Device: Monitor

4.2 Software Requirements

- Operating System: Windows 10/Linux
- Programming Language: Java
- Frontend Technologies: React, JSX, Bootstrap, JavaScript
- Backend Technology: Java, Spring Boot, REST APIs
- Database: MySQL
- Web Server: Apache Tomcat
- Development Tools: Eclipse / IntelliJ IDEA, Postman, VS Code
- Browser: Google Chrome / Mozilla Firefox

CHAPTER-5

SOFTWARE DESIGN

5.1 System Architecture

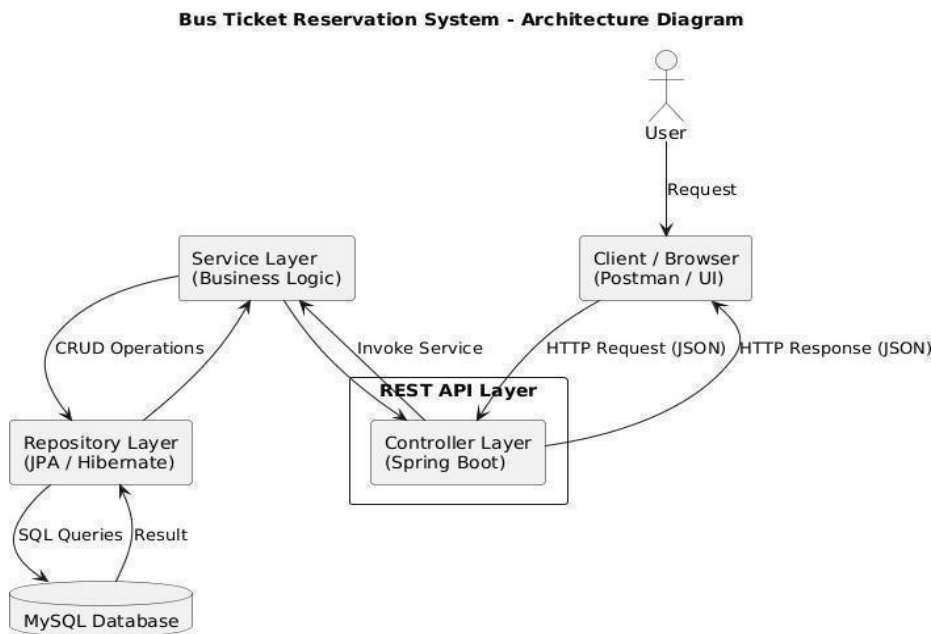


Fig 5.1 System Architecture

5.2 UML Diagrams

5.2.1 Class Diagram

The class diagram is used to represent the static structure of the Bus Ticket Reservation System and to provide a detailed design view of the system. It refines the use case diagram by identifying the key classes involved in the system and defining the relationships among them. The class diagram classifies the system components into a set of interrelated classes such as User, Bus, Journey, Ticket, and Booking.

The relationships between classes can be either an “is-a” relationship (inheritance) or a “has-a” relationship (association). Each class in the diagram is responsible for performing specific functionalities of the system. These functionalities are represented as methods of the respective classes. Additionally, each class contains attributes that store important data related to bus journeys, ticket bookings, and users.

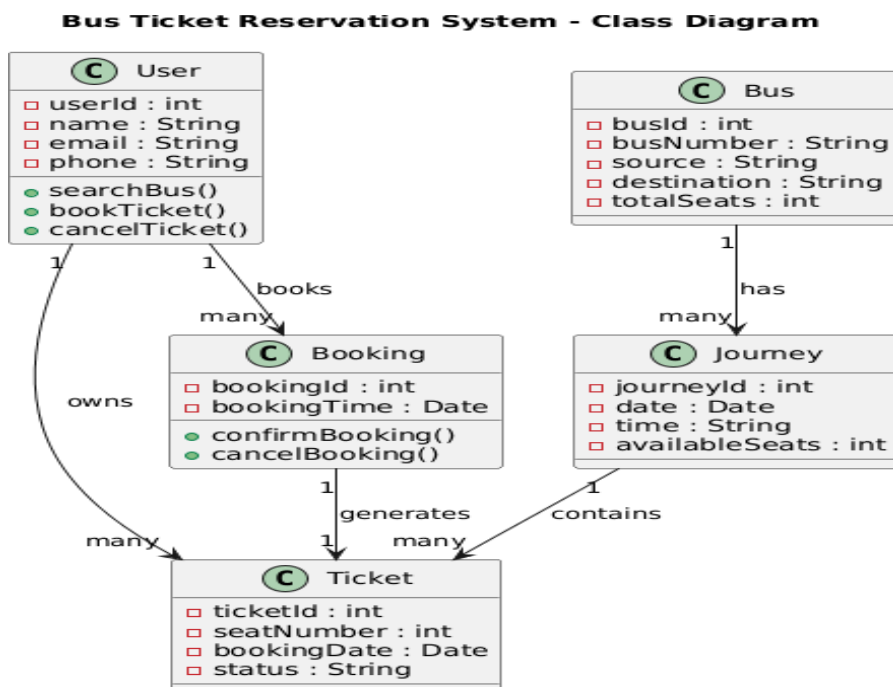


Fig 5.2 Class Diagram

5.2.2 Use Case Diagram

A Use Case Diagram in the Unified Modeling Language (UML) represents the functional behavior of the Bus Ticket Reservation System from the user's perspective. It provides a graphical overview of the system's functionalities by identifying the actors, the use cases, and the interactions between them. Each use case represents a specific function that the system performs to achieve a user goal.

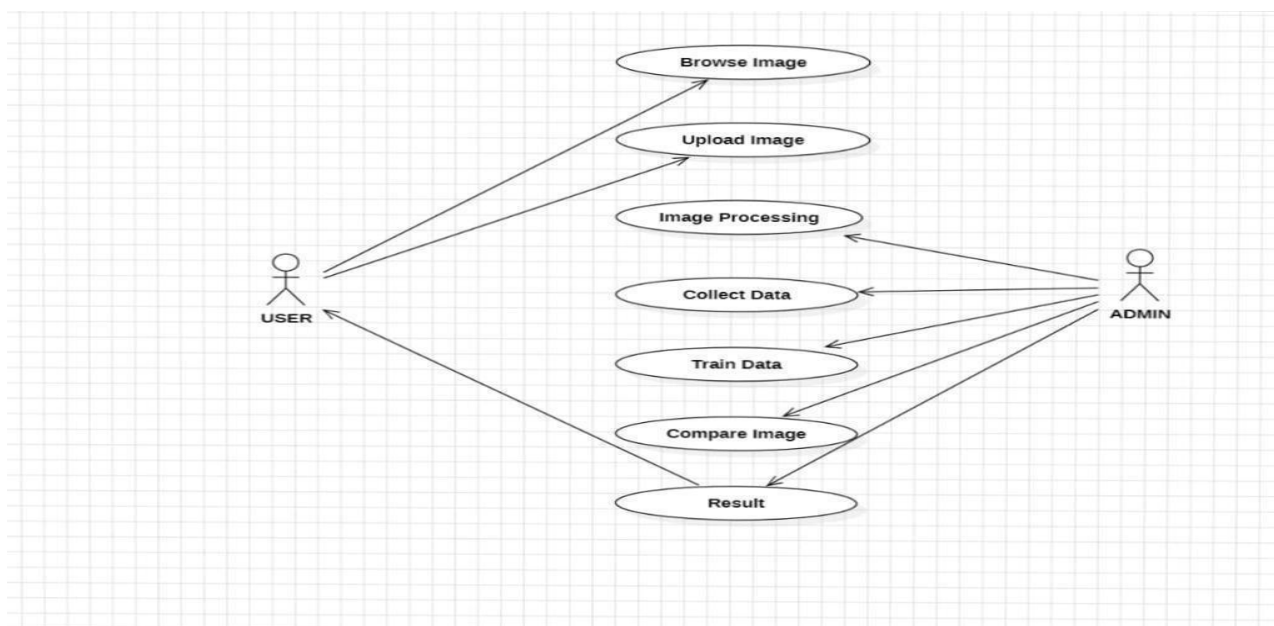


Fig 5.3 Use Case Diagram

5.2.3 Sequence Diagram

A Sequence Diagram represents the interaction between different objects in the Bus Ticket Reservation System in a time-ordered manner. It illustrates how various components of the system communicate with each other step by step to complete a specific task such as searching buses, booking tickets, or cancelling reservations. The sequence diagram focuses on the order in which messages are exchanged between system components.

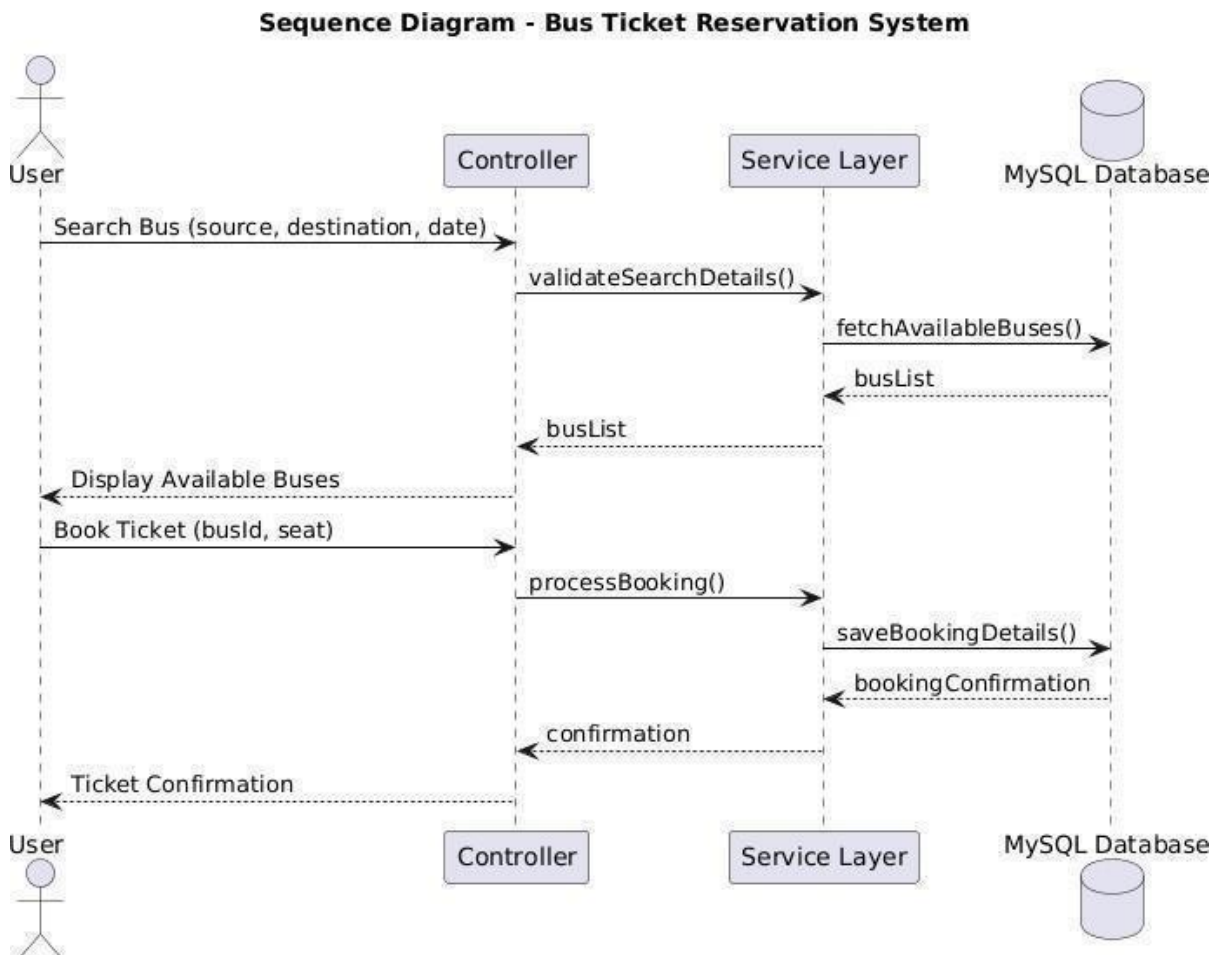


Fig 5.4 Sequence Diagram

CHAPTER-6

SYSTEM ENVIRONMENT

6.1 What is Java?

- Java is a widely used, high-level, object-oriented programming language designed to be secure, robust, and platform independent. It is one of the most preferred languages for developing enterprise-level and web-based applications.
- Java follows the principle of Write Once, Run Anywhere (WORA), which means that Java programs compiled into bytecode can run on any system that has a Java Virtual Machine (JVM), regardless of the underlying hardware or operating system.
- Java supports core object-oriented programming concepts such as encapsulation, inheritance, abstraction, and polymorphism, which help in building modular, reusable, and maintainable software systems.
- Java is extensively used by leading technology companies such as Google, Amazon, Netflix, Oracle, and IBM, making it a reliable and industry-accepted programming language.

The biggest strength of Java lies in its robust ecosystem and extensive standard libraries, which are widely used for:

- Web application development
- Enterprise-level systems
- Distributed and cloud-based applications
- Database-driven applications
- Backend services and APIs
- Secure transaction processing

Advantages of Java :-

Let's see how Java dominates over other languages.

1. Extensive Libraries

Java provides a rich set of built-in libraries and APIs such as Collections Framework, JDBC, Networking, Concurrency, Security, XML/JSON processing, and GUI libraries. These libraries reduce development time by offering ready-made solutions for common programming tasks, eliminating the need to write code from scratch.

2. Platform Independent (WORA)

Java follows the principle of Write Once, Run Anywhere (WORA). Java programs are compiled into bytecode, which can run on any system that has a Java Virtual Machine (JVM), making highly portable.

3. Robust and Secure

Java is designed with strong memory management, automatic garbage collection, exception handling, and type checking. Its security features such as bytecode verification, sandboxing, and absence of pointers make it suitable for secure and large-scale applications. High Performance.

4. Enterprise and Web Application Support

Java is widely used for enterprise applications through technologies like Spring, Hibernate, Jakarta EE, and Microservices architecture. It is a preferred choice for banking, e-commerce, and large distributed systems.

5. Object-Oriented

Java is a purely object-oriented language (except for primitive data types). It supports core OOP concepts such as inheritance, polymorphism, encapsulation, and abstraction, enabling developers to model real-world problems effectively.

6. Readable and Maintainable Code

Java enforces a structured syntax, which makes programs easy to read, debug, and maintain. Its strict typing and clear structure help reduce runtime errors and improve long-term maintainability of large projects.

7. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

8. Free and Open-Source

Java is open source and freely available through OpenJDK. Developers can use, modify, and distribute Java without licensing costs, making it accessible for students, researchers.

9. Portable

Java is highly scalable and is used in applications ranging from small desktop programs to large distributed systems. Its stability and reliability make it a trusted language for mission-critical applications.

10. Interpreted

Java combines both compilation and interpretation. Source code is compiled into bytecode, and the JVM interprets or compiles it at runtime.

6.2 Advantages of Java over Other Languages

1. High Performance

Java uses Just-In-Time (JIT) compilation, which improves runtime performance. This means that Java programs run efficiently and execute faster compared to many other languages. Its optimized memory management and garbage collection also help applications perform smoothly under heavy load.

2.Strong Community Support

Java has a large developer community and extensive documentation. Beginners and professionals alike can find tutorials, forums, and libraries that make coding easier. The active community ensures continuous updates, support, and solutions for almost any problem developers face.

3. Java is for Everyone

Java is widely used in enterprise applications due to its stability and scalability. Organizations rely on Java for building secure, high-performance, and cross-platform applications. Its ability to handle complex business logic, large datasets, and concurrent users makes it ideal for long- term projects and mission-critical systems.

6.3 Disadvantages of Java

1. Verbose Syntax

Java programs often require writing more lines of code compared to languages like Python. This can make simple tasks appear more complex and increases development time, especially for beginners and small projects.

2. Slower Startup Time

Java applications, particularly large ones, may take longer to start because of the initialization of the Java Virtual Machine (JVM). For lightweight scripts or quick tasks, this startup delay can be noticeable.

3. Higher Memory Consumption

Java programs typically consume more memory due to the JVM and garbage collection mechanisms. This can be a limitation for applications running on devices with limited system resources.

4. Limited Low-Level Programming

Java abstracts hardware-level details, which makes it unsuitable for low-level system programming where direct memory or hardware control is required. Languages such as C and C++ are better suited for such tasks.

5. Slower than Native Compiled Languages

Although Java offers good performance, it is generally slower than programs written in native compiled languages like C or C++. The additional JVM layer introduces some performance overhead.

6.4 History of Java

Java is a general-purpose, object-oriented programming language that was designed to be portable, secure, and robust. It was originally developed in the early 1990s by James Gosling and his team at Sun Microsystems as part of a project known as the Green Project. The goal of this project was to create a programming language that could run on different electronic devices without modification.

Initially, the language was named Oak, after an oak tree outside Gosling's office. Later, it was renamed

Java, inspired by the coffee that the development team frequently consumed during long coding sessions.

Java was officially released in 1995 and quickly gained popularity because of its platform independence, achieved through the Java Virtual Machine (JVM). The JVM allows Java programs to be executed on any system that supports the JVM, regardless of the underlying hardware or operating system.

Over time, Java evolved into one of the most widely used programming languages in the world. It is now maintained by Oracle and the open-source community through OpenJDK and continues to be widely used for web applications, enterprise systems, mobile development, and cloud-based platforms.

6.5 What is Spring Boot?

Spring Boot is a framework built on top of the Spring Framework that simplifies the development of Java-based applications. It reduces the complexity of configuration by providing automatic setup, embedded servers, and sensible default settings, allowing developers to focus more on application logic rather than environment configuration.

Spring Boot enables developers to create standalone, production-ready applications with minimal effort. It supports the development of RESTful web services, database integration, security implementation, and application monitoring.

Features such as embedded Tomcat or Jetty servers allow applications to run without requiring external web servers. This makes development, testing, and deployment faster and easier.

Because of its simplicity, flexibility, and integration capabilities, Spring Boot has become a popular choice for building modern, scalable, and maintainable enterprise and web applications.

6.6 What is REST API

REST (Representational State Transfer) is an architectural style used for designing web services. REST APIs allow communication between client and server using standard HTTP methods such as GET, POST, PUT, and DELETE.

REST APIs exchange data in lightweight formats like JSON, making them fast and easy to use. In the Bus Ticket Reservation System, REST APIs are used to perform operations such as searching buses, booking tickets, cancelling reservations, and retrieving booking details. RESTful services ensure scalability, flexibility, and smooth communication between system components

Need for Spring Boot

Spring Boot is required in this project to simplify and accelerate the development of the Bus Ticket Reservation System. Traditional Spring applications require extensive XML configurations and manual setup, which increases development time and complexity. Spring Boot eliminates these issues by providing auto-configuration and predefined defaults, allowing developers to focus on business

logic rather than configuration.

Spring Boot makes it easy to develop RESTful web services, which are essential for handling operations such as searching buses, booking tickets, and cancelling reservations. It includes an embedded server (Tomcat), removing the need for external server deployment. This makes the application lightweight, portable, and easy to run.

Additionally, Spring Boot supports seamless integration with databases using JPA and Hibernate, enabling efficient data persistence and management. Its layered architecture improves code organization, scalability, and maintainability. Due to these advantages, Spring Boot is an ideal framework for building a reliable, secure, and scalable backend system for this project.

6.7 Database Environment – MySQL

- MySQL is a relational database management system (RDBMS) used to store and manage structured data. It organizes data into tables with rows and columns, ensuring data integrity and consistency.
- In this project, MySQL is used to store information related to buses, journeys, users, and ticket bookings. It supports SQL queries for data insertion, retrieval, update, and deletion. MySQL is reliable, secure, and widely used in web applications, making it suitable for this Bus Ticket Reservation System.

6.8 Development Tools Used

- Java JDK – For writing and running Java applications
- Spring Boot – For backend application development
- MySQL – For database management
- Postman – For testing REST APIs
- IDE (IntelliJ IDEA / Eclipse) – For coding and debugging
- Maven – For dependency management and project build

CHAPTER-7

TECHNOLOGIES USED

The Bus Ticket Reservation System is designed using a modular approach to ensure better organization, scalability, and ease of maintenance. Each module is responsible for a specific set of functionalities, allowing independent development and testing. This modular design also helps in identifying errors easily and enhances system reliability. The major modules implemented in this project are the User Module, Admin Module, and Booking Module.

7.1 User Module

- The User Module enables passengers to interact with the system and access its core services. Through this module, users can search for available buses by providing journey details such as source, destination, date, and time. The system displays relevant bus information including timings, seat availability, and fare details, helping users make informed booking decisions.
- In addition to searching, the User Module allows users to book tickets and cancel existing reservations. All user actions are processed through RESTful APIs, ensuring secure and efficient communication between the client and server. This module focuses on providing a simple and user- friendly experience while maintaining accuracy and reliability in booking operations.

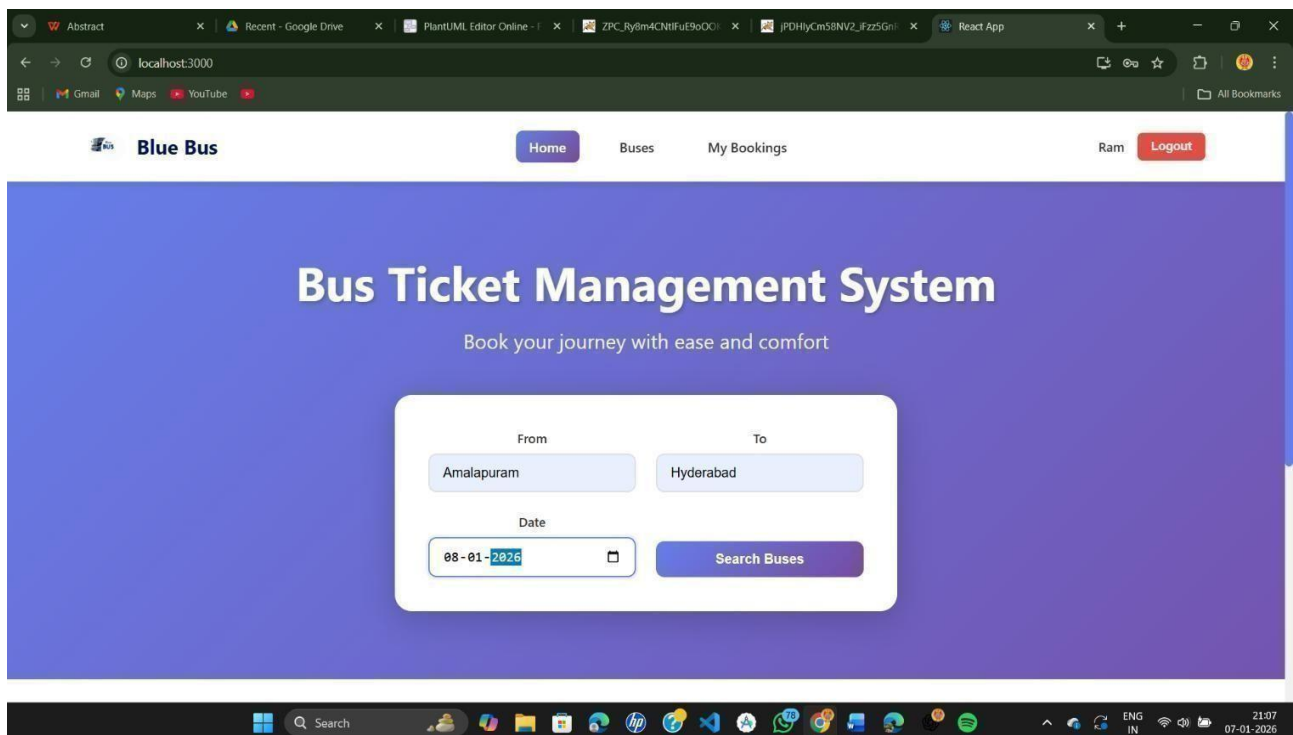


Fig 7.1 Shows User Interface for searching buses

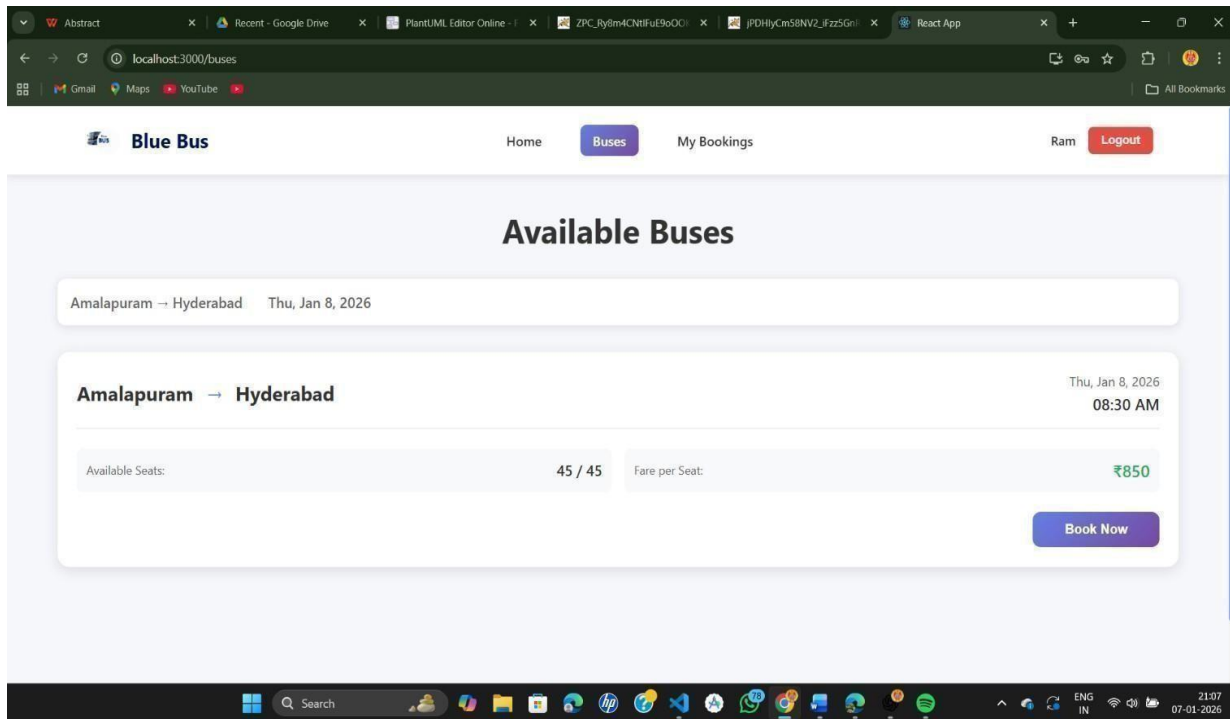


Fig 7.2 Displays the list of available buses.

7.2 Admin Module

- The Admin Module is designed to manage and control system data. Administrators can add new bus journey details, update schedules, modify seat availability, and remove outdated or incorrect records. This ensures that users always receive accurate and up-to-date information while searching for buses.
- Furthermore, the Admin Module allows administrators to monitor bookings and maintain data consistency across the system. By centralizing control, this module reduces data redundancy and improves system efficiency. Proper validation checks are implemented to prevent incorrect data entry, making the system more reliable.

7.3 Booking Module

- The Booking Module is the core component of the Bus Ticket Reservation System. It handles ticket booking requests by validating user input, checking seat availability, and generating booking confirmations. Once a booking is successful, the system updates the database and assigns a unique ticket ID to the user.
- This module also manages ticket cancellations by releasing reserved seats and updating booking status. It ensures consistency in booking data and prevents issues such as overbooking. The Booking Module plays a crucial role in maintaining transaction accuracy and smooth system operation.

The screenshot shows the 'Blue Bus' website interface for booking a ticket. The browser address bar shows 'localhost:3000/booking/1'. The page has a navigation bar with 'Home', 'Buses', and 'My Bookings' links, and a user profile 'Ram' with a 'Logout' button. The main content area is divided into two columns:

- Journey Details:**
 - Route: Amalapuram → Hyderabad
 - Date: Thursday, January 8, 2026
 - Time: 08:30 AM
 - Available Seats: 45
 - Fare per Seat: ₹850
- Passenger Information:**
 - Name: Ram
 - Email: ram@gmail.com
 - Phone: N/A
 - Number of Seats: 1 (Maximum 45 seats available)
 - Seats: 1 × ₹850
 - Total Fare: ₹850

Fig 7.3 Illustrates the ticket booking process.

The screenshot shows the 'Blue Bus' website interface for 'My Bookings'. The browser address bar shows 'localhost:3000/my-bookings'. The page has a navigation bar with 'Home', 'Buses', and 'My Bookings' links, and a user profile 'Ram' with a 'Logout' button. The main content area is titled 'My Bookings' and displays a confirmed booking for Journey #1:

Journey Details		Passenger Information		Booking Information	
Route:	Amalapuram → Hyderabad	Name:	Ram	Seats:	2
Journey Date:	Thu, Jan 8, 2026	Email:	ram@gmail.com	Total Fare:	₹1700
Journey Time:	08:30 AM	Phone:	8654567689	Booking Date:	Jan 7, 2026, 09:15 PM

A 'Cancel Booking' button is visible at the bottom right of the booking details card.

Fig 7.4 Shows Ticket Confirmation Scree

CHAPTER-8

TESTING

Testing is a critical phase in the software development life cycle that ensures the application works as expected and meets user requirements. In the Bus Ticket Reservation System, testing is performed to verify the correctness, reliability, and performance of all modules. It helps in identifying defects early and ensures that the system functions smoothly under various conditions.

A systematic testing approach is followed to validate individual components as well as the complete system. Different testing techniques are applied to ensure that each feature such as bus search, ticket booking, and cancellation works correctly. Testing also helps in improving system stability and reducing the chances of failure after deployment.

8.1 System Testing

System testing involves testing the complete and integrated Bus Ticket Reservation System to verify that it satisfies all specified requirements. In this phase, the system is tested as a whole to ensure that all modules work together correctly.

During system testing, real-time scenarios are considered, such as searching for buses, booking tickets, and cancelling reservations. This testing ensures that the data flows correctly between the controller, service, and database layers. System testing helps in validating the overall behavior of the application and confirms that the system is ready for deployment.

8.2 Types of Testing

8.2.1 Unit Testing

Unit testing is performed to test individual components or modules of the application independently. In this project, unit testing is applied to service methods and repository functions to ensure that each unit performs its intended task correctly.

Unit tests help in identifying logical errors at an early stage of development. They improve code quality and make debugging easier.

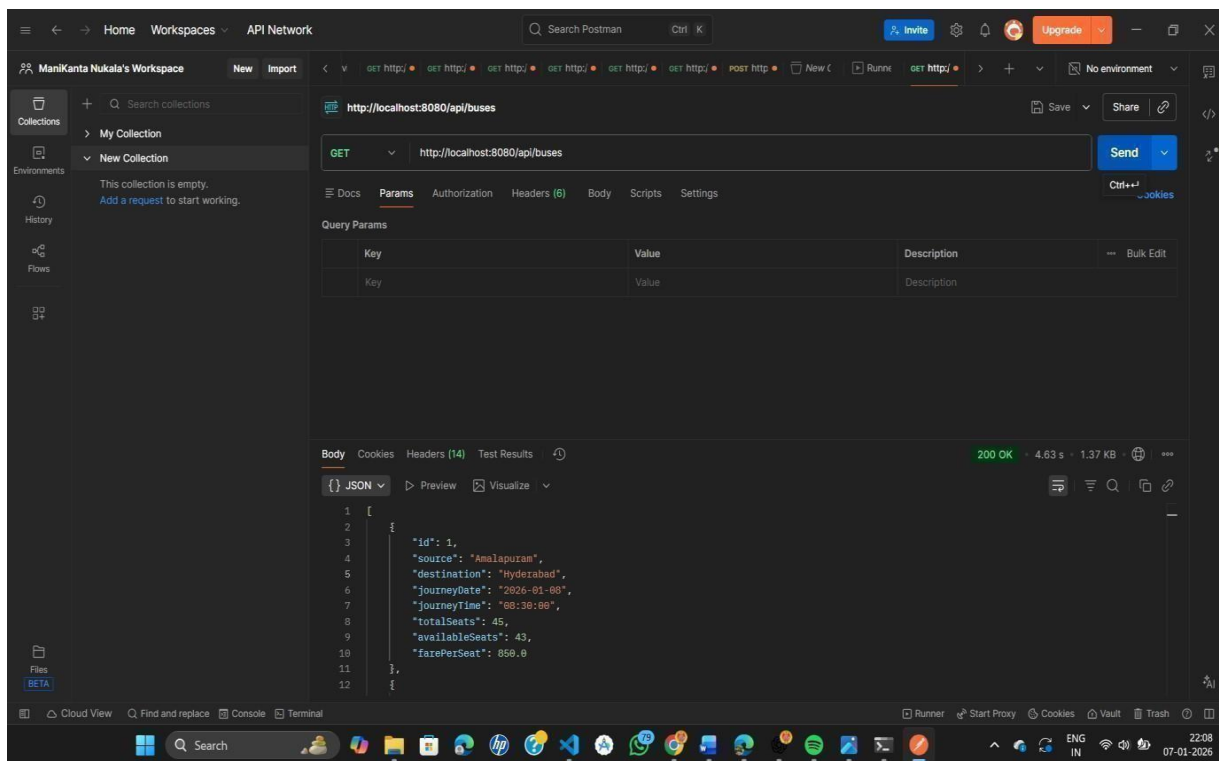


Fig 8.1 Output of Unit Testing

8.2.2 Integrated Testing

Integration testing is conducted to verify the interaction between multiple modules of the system. In the Bus Ticket Reservation System, integration testing ensures that controllers, services, and the database interact correctly.

This testing validates data flow between layers and ensures that combined modules produce expected results. It helps in detecting errors related to incorrect API communication or database integration.

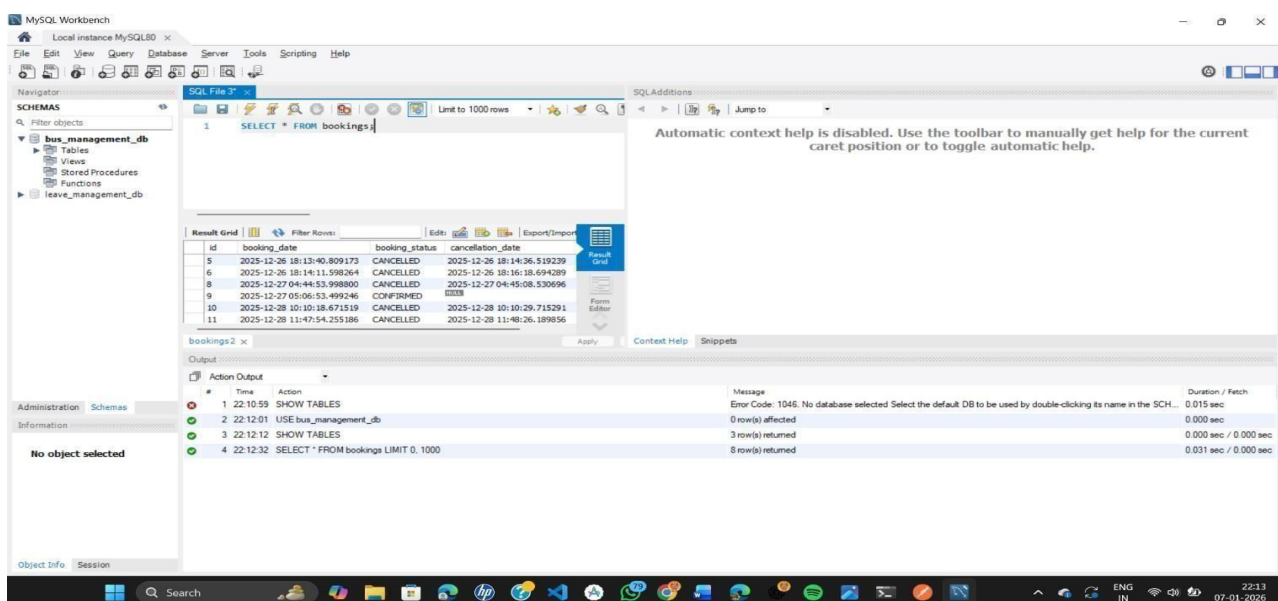


Fig 8.2 Represents the Integration Testing Flow.

8.2.4 Functional test

Functional testing verifies that the system functions according to the specified requirements. Features such as bus search, ticket booking, and cancellation are tested to ensure correct functionality.

Functional testing is centered on the following items:

1. Valid Input

Identified classes of valid input must be accepted by the system. In this project, valid inputs such as correct source and destination names, valid journey dates, and available seat numbers are tested. The system should process these inputs successfully and return correct results.

2. Invalid Input

Identified classes of invalid input must be rejected. Inputs such as incorrect locations, invalid dates, empty fields, or unavailable seat numbers are tested to ensure that the system handles errors properly and displays appropriate error messages.

3. Functions

All identified system functions must be exercised during testing. Core functionalities such as searching for buses, booking tickets, cancelling tickets, and viewing booking status are tested to ensure they work as intended.

4. Output

Identified classes of application outputs must be exercised. Outputs such as available bus lists, booking confirmations, cancellation status messages, and error responses are verified for correctness and consistency.

5. Systems Procedures

Interfacing systems or procedures must be invoked correctly. In this project, interactions between the REST API, service layer, and database are tested to ensure smooth data flow and correct execution of procedures.

8.2.4 White Box Testing

White box testing is a testing technique that focuses on validating the internal logic, code structure, and execution flow of the application. In this approach, the tester has complete knowledge of the source code and examines how different components interact at the code level. It helps ensure that the system functions correctly under all possible conditions.

In the Bus Ticket Reservation System, white box testing is primarily applied to the service layer and repository layer. Methods responsible for searching buses, validating seat availability, processing ticket bookings, and handling cancellations are tested in detail. Control structures such as loops, conditional statements, and exception handling blocks are examined to ensure correct execution.

White box testing improves code quality by increasing code coverage and identifying hidden logical errors. It also helps developers optimize performance and maintain clean, maintainable code. By performing white box testing, the system becomes more reliable and less prone to unexpected failures during runtime.

8.2.5 Black Box Testing

Black box testing is a functional testing technique that evaluates the system without any knowledge of its internal code structure. The tester focuses only on providing input and observing the output, ensuring that the application behaves as expected from the user's perspective.

In this project, black box testing is performed on the REST API endpoints using tools such as Postman. APIs related to bus search, ticket booking, and cancellation are tested with both valid and invalid inputs. This ensures that the system returns correct responses, handles errors gracefully, and maintains data consistency.

Black box testing helps validate system requirements and user expectations. It ensures that the Bus Ticket Reservation System is user-friendly, robust, and reliable. This type of testing is essential to confirm that the application delivers correct functionality without exposing internal implementation details.

8.2.6 White Box Testing

White box testing is a structural testing technique that evaluates the system by examining its internal code, logic, and control flow. The tester has knowledge of the program implementation and verifies whether the code works correctly for different conditions, loops, and execution paths.

In this project, white box testing is performed on backend modules and REST API service logic such as user authentication, bus search processing, seat availability validation, booking confirmation, payment status updates, and cancellation/refund handling. The internal methods are tested to ensure correct decision making (if/else conditions), proper loop execution, and accurate database operations for both normal and edge case scenarios.

White box testing helps in identifying logical errors, unreachable code, and incorrect data handling within the application. It improves code quality, ensures reliability of critical booking workflows, and confirms that the Bus Ticket Reservation System functions correctly at the implementation level.

• Test Strategy and Approach

Unit testing is performed using manual testing and API-based testing techniques. Each API endpoint is tested individually using tools such as Postman to verify correct responses. Functional tests are written and executed in detail to ensure that individual units behave as expected.

Acceptance Testing

Acceptance testing is the final testing phase carried out to ensure that the system meets user requirements and expectations. User Acceptance Testing (UAT) involves validating the system from the end-user's perspective to confirm that it is ready for deployment.

In the Bus Ticket Reservation System, acceptance testing verifies that users can successfully search for buses, book tickets, and cancel reservations without any issues. This testing ensures that the system

fulfills all functional requirements and provides a satisfactory user experience. Test Results

All acceptance test cases were passed successfully. The system met the specified requirements, and no defects were encountered during testing.

Test Case– 1: LOGIN FUNCTIONALITY

Test Case ID	TC_01
Function	Login
Input Data	Valid Username and Password
Expected Result	System should validate the user and allow login
Actual Result	User credentials validated successfully
Priority	High

Test Case– 2: SEARCH BUS

Test case for Search Buses:

FUNCTION:	Search Bus
Input Data	Source ,Destination, Date
Expected Result	Display available buses based on search criteria
Actual Result	Matching buses displayed successfully
Status	Pass

Test Case – 3: TICKET BOOKING

FUNCTION:	Book Ticket
Input Data	Selected bus and seat.
Expected Result	Ticket should be booked and confirmation generated
Actual Result	Ticket booked successfully with Confirmation
Status	Pass

Test Case – 4 : INVALID LOGIN

Function	Invalid Login
Input Data	Invalid Username and password
Expected Result	System should display error
Actual Result	Error message displayed correctly
Priority	Medium
Status	Pass

CHAPTER – 9

IMPLEMENTATION

Implementation is the phase in which the system design is converted into a working application. In this stage, all planned modules are developed, integrated, and tested to ensure that the system functions according to requirements. The Bus Ticket Reservation System is implemented using Java, Spring Boot, REST APIs, and MySQL.

The implementation focuses on backend development, where RESTful APIs handle user requests such as searching buses, booking tickets, and cancelling reservations. The system follows a layered architecture, which improves code organization, maintainability, and scalability.

9.1 Methodology

The development of the Bus Ticket Reservation System follows a modular and incremental approach. Each module is implemented separately and then integrated with other modules. This approach helps in early detection of errors and ensures smooth system integration.

The methodology used in this project includes the following steps

1. Requirement Understanding

Functional and non-functional requirements were analyzed to understand system behavior and user needs.

2. Design Implementation

- The model is trained using the prepared dataset.
- The model architecture is chosen based on the complexity and size of the dataset.
- The training process is monitored using performance metrics like mAP (mean Average Precision), loss curves, and precision-recall.

3. Backend Development

- Spring Boot is used to create RESTful APIs. Controllers handle incoming HTTP requests, services contain business logic, and repositories interact with the MySQL database.

4. Database Integration

- MySQL is integrated using JPA and Hibernate to store and retrieve bus, booking, and user data efficiently.

5. Testing and Validation

- The processed image with detection boxes is converted to base64 and rendered in the browser using matplotlib and Django's templating engine.
- Detection results are displayed on the UserScreen.html page with bounding boxes and class names.

6. Training Performance Visualization

APIs are tested using Postman to verify correctness of responses and data consistency.

9.2 Source Code

The source code of the Bus Ticket Reservation System is organized using a layered structure, ensuring separation of concerns

a) Controller Layer

The controller layer handles HTTP requests and maps them to appropriate services using REST annotations such as `@GetMapping`, `@PostMapping`, and `@DeleteMapping`.
package com.mybus.busManagement.controller;

```
import com.mybus.busManagement.dto.BookingRequestDTO;
import com.mybus.busManagement.dto.BookingResponseDTO;
import com.mybus.busManagement.entity.User;
import com.mybus.busManagement.exception.BusinessException;
import com.mybus.busManagement.service.AuthService;
import com.mybus.busManagement.service.BookingService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.Valid;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
```

```
import org.springframework.security.core.Authentication;
```

```
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

```
import java.util.Map; @RestController @RequestMapping("/api/bookings")
@CrossOrigin(origins = "*")
public class BookingController { @Autowired
private BookingService bookingService; @Autowired
private AuthService authService;

private User getCurrentUser(HttpServletRequest request) {
String email = (String) request.getAttribute("username");
// If not in request attribute, try to get from SecurityContext if (email == null) {
Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

if (authentication != null && authentication.isAuthenticated() && authentication.getPrincipal()
instanceof String) {
```



```

email = (String) authentication.getPrincipal();

}

}

if (email == null) {
throw new BusinessException("User not authenticated. Please login again.");
}
return authService.getUserByEmail(email);

}

```

@PostMapping

```

public ResponseEntity<BookingResponseDTO> createBooking(

@Valid @RequestBody BookingRequestDTO bookingRequestDTO, HttpServletRequest request) {
User user = getCurrentUser(request);

BookingResponseDTO bookingResponse = bookingService.createBooking(bookingRequestDTO,
user);
return new ResponseEntity<>(bookingResponse, HttpStatus.CREATED);
}

```

@GetMapping

```

public ResponseEntity<List<BookingResponseDTO>> getAllBookings(HttpServletRequest request)
{
User user = getCurrentUser(request); List<BookingResponseDTO> bookings; if (user.isAdmin()) {
bookings = bookingService.getAllBookings();

} else {

bookings = bookingService.getBookingsByUser(user);

}

return new ResponseEntity<>(bookings, HttpStatus.OK);

}

```

@GetMapping("/{id}")

```

public ResponseEntity<BookingResponseDTO> getBookingById(@PathVariable Long id)
{ BookingResponseDTO booking = bookingService.getBookingById(id);
return new ResponseEntity<>(booking, HttpStatus.OK);
}

```

```
}

@GetMapping("/email/{email}")

public ResponseEntity<List<BookingResponseDTO>>
getBookingsByEmail(@PathVariable String email) {
    List<BookingResponseDTO> bookings = bookingService.getBookingsByEmail(email);
    return new ResponseEntity<>(bookings, HttpStatus.OK);
}

@PutMapping("/{id}/cancel")

public ResponseEntity<BookingResponseDTO> cancelBooking( @PathVariable Long id,
HttpServletRequest request) { User user = getCurrentUser(request);
BookingResponseDTO bookingResponse = bookingService.cancelBooking(id, user);
return new ResponseEntity<>(bookingResponse, HttpStatus.OK);
}

@GetMapping("/confirmed")

public ResponseEntity<List<BookingResponseDTO>>
getConfirmedBookings() { List<BookingResponseDTO>
bookings = bookingService.getConfirmedBookings();
return new ResponseEntity<>(bookings, HttpStatus.OK);
}

@GetMapping("/cancelled")

public ResponseEntity<List<BookingResponseDTO>>
getCancelledBookings() { List<BookingResponseDTO>
bookings = bookingService.getCancelledBookings();
return new ResponseEntity<>(bookings, HttpStatus.OK);

}

}
```

b) Service Layer

The service layer contains the business logic of the application. It processes requests, validates data, and communicates with the repository layer.

Code:

```
package com.mybus.busManagement.service;

import com.mybus.busManagement.dto.BookingRequestDTO;
import com.mybus.busManagement.dto.BookingResponseDTO;
```

```
import com.mybus.busManagement.entity.Booking;
import com.mybus.busManagement.entity.Booking.BookingStatus;
import com.mybus.busManagement.entity.Bus;
import com.mybus.busManagement.entity.User;

import com.mybus.busManagement.exception.BusinessException;
import com.mybus.busManagement.exception.ResourceNotFoundException;
import com.mybus.busManagement.repository.BookingRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional; import java.time.LocalDateTime;
import java.util.List;

import java.util.stream.Collectors; @Service
@Transactional

public class BookingService { @Autowired
private BookingRepository bookingRepository; @Autowired
private BusService busService; @Autowired
private AuthService authService;

public BookingResponseDTO createBooking(BookingRequestDTO bookingRequestDTO, User user)
{
    Bus bus = busService.getBusEntityById(bookingRequestDTO.getBusId());
    if (bus.getAvailableSeats() < bookingRequestDTO.getNumberOfSeats())
    { throw new BusinessException("Insufficient seats available. Available seats: " +
    bus.getAvailableSeats());

    }

    Booking booking = new Booking( bus,
    user, user.getName(), user.getEmail(), user.getPhone(),
    bookingRequestDTO.getNumberOfSeats()

    );

    bus.setAvailableSeats(bus.getAvailableSeats() - bookingRequestDTO.getNumberOfSeats());
    Booking savedBooking = bookingRepository.save(booking);

    return convertToResponseDTO(savedBooking);

}

public List<BookingResponseDTO> getAllBookings() { return bookingRepository.findAll().stream()
.map(this::convertToResponseDTO)
```

```
.collect(Collectors.toList());

}

public BookingResponseDTO getBookingById(Long id)
{ Booking booking = bookingRepository.findById(id)
.orElseThrow(() -> new ResourceNotFoundException("Booking not found with id: " +
id));

return convertToResponseDTO(booking);

}

public List<BookingResponseDTO> getBookingsByUser(User user)
{ List<Booking> bookings = bookingRepository.findById(user.getId());
return bookings.stream()
.map(this::convertToResponseDTO)

.collect(Collectors.toList());

}

public List<BookingResponseDTO> getBookingsByEmail(String email)
{ List<Booking> bookings = bookingRepository.findByPassengerEmail(email);
return bookings.stream()
.map(this::convertToResponseDTO)

.collect(Collectors.toList());

}

public BookingResponseDTO cancelBooking(Long id, User user)
{ Booking booking = bookingRepository.findByIdAndBookingStatus(id,
BookingStatus.CONFIRMED)

.orElseThrow(() -> new ResourceNotFoundException( "Booking not found with id: " + id + " or
already cancelled"));
if (!user.isAdmin() && !booking.getUser().getId().equals(user.getId())) {

throw new BusinessException("You are not authorized to cancel this booking");

}

booking.setBookingStatus(BookingStatus.CANCELLED);
booking.setCancellationDate(LocalDateTime.now());
Bus bus = booking.getBus();

bus.setAvailableSeats(bus.getAvailableSeats() + booking.getNumberOfSeats());
```

```

Booking cancelledBooking = bookingRepository.save(booking);
return convertToResponseDTO(cancelledBooking);

}
public List<BookingResponseDTO> getConfirmedBookings()
{
    List<Booking> bookings =
    bookingRepository.findByBookingStatus(BookingStatus.CONFIRMED);
    return bookings.stream()
    .map(this::convertToResponseDTO)
    .collect(Collectors.toList());
}
public List<BookingResponseDTO> getCancelledBookings() { List<Booking> bookings =
    bookingRepository.findByBookingStatus(BookingStatus.CANCELLED); return bookings.stream()
    .map(this::convertToResponseDTO)
    .collect(Collectors.toList());

}

private BookingResponseDTO convertToResponseDTO(Booking booking)
{ return new BookingResponseDTO(
    booking.getId(), booking.getBus().getId(),
    booking.getBus().getSource(), booking.getBus().getDestination(),
    booking.getBus().getJourneyDate(), booking.getBus().getJourneyTime(),
    booking.getPassengerName(), booking.getPassengerEmail(),
    booking.getPassengerPhone(), booking.getNumberOfSeats(),
    booking.getTotalFare(), booking.getBookingStatus().toString(),
    booking.getBookingDate(), booking.getCancellationDate()
);
}
}

```

c) Repository Layer

The repository layer interacts directly with the database using JPA repositories. It performs CRUD operations on database tables.

Code:

```

package com.mybus.busManagement.repository; import com.mybus.busManagement.entity.Booking;
import com.mybus.busManagement.entity.Booking.BookingStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List; import java.util.Optional; @Repository
public interface BookingRepository extends JpaRepository<Booking, Long>
{ List<Booking> findByPassengerEmail(String passengerEmail);
Optional<Booking> findByIdAndBookingStatus(Long id, BookingStatus status);
List<Booking> findByBookingStatus(BookingStatus status);
List<Booking> findByBusId(Long busId);
List<Booking> findByBusIdAndBookingStatus(Long busId, BookingStatus status);
List<Booking> findByUserId(Long userId);
}

```

d) Users

```

USERS: package com.mybus.busManagement.entity;
import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import java.time.LocalDateTime;
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank(message = "Name is required")
    @Column(nullable = false)
    private String name;
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    @Column(nullable = false, unique = true)
    private String email;
    @NotBlank(message = "Password is required")
    @Column(nullable = false)
    private String password;
    @NotBlank(message = "Phone is required")
    @Column(nullable = false)
    private String phone;
    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;
    @Column(name = "role", nullable = false)
    private String role; // "USER" or "ADMIN"
    public User() {
        this.createdAt = LocalDateTime.now();
        this.role = "USER";
    }
    public User(String name, String email, String password, String phone) {
        this.name = name;
        this.email = email;
        this.password = password;
        this.phone = phone;
        this.createdAt = LocalDateTime.now();
        this.role = "USER";
    }
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
}

```

```
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getEmail() {  
    return email;  
}  
public void setEmail(String email) {  
    this.email = email;  
}  
public String getPassword() {  
    return password;  
}  
public void setPassword(String password) {  
    this.password = password;  
}  
public String getPhone() {  
    return phone;  
}  
public void setPhone(String phone) {  
    this.phone = phone;  
}  
public LocalDateTime getCreatedAt() {  
    return createdAt;  
}  
public void setCreatedAt(LocalDateTime createdAt) {  
    this.createdAt = createdAt;  
}  
public String getRole() {  
    return role;  
}  
public void setRole(String role) {  
    this.role = role;  
}  
public boolean isAdmin() {  
    return "ADMIN".equalsIgnoreCase(this.role);  
}  
}
```

CHAPTER –10

IMPLEMENTATION, RESULTS AND SCREENSHOTS

10.1 Introduction

This chapter describes the implementation details, system execution, and output results of the proposed Bus Ticket Reservation System.

The system is developed to automate the process of bus search, seat booking, ticket reservation, and management, reducing manual effort and improving efficiency.

The application is implemented using:

- Java for backend processing
- MySQL for database management
- HTML, CSS, JavaScript for frontend
- Apache Tomcat server for deployment

10.2 System Modules

The Bus Ticket Reservation System consists of the following modules:

10.2.1 User Login Module

This module allows users and administrators to log in using a valid username and password. Authentication ensures secure access to system features.

Functions:

- User authentication
- Role-based access (Admin/User)
- Secure login validation

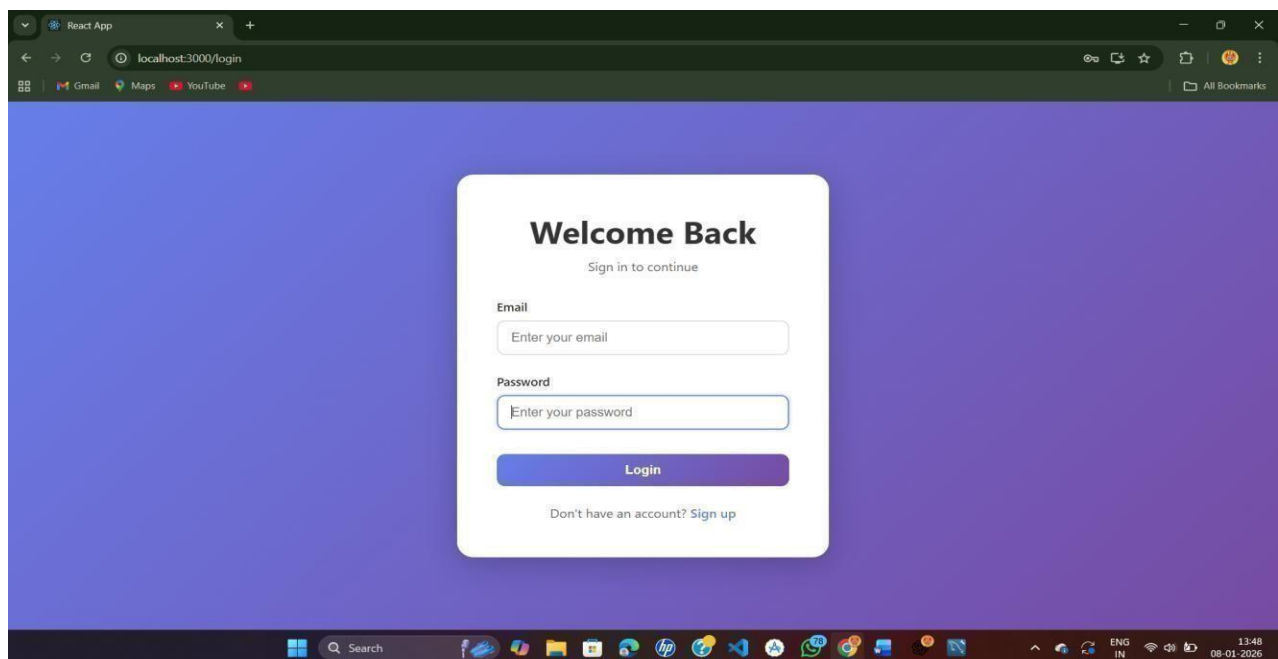


Fig 10.1 User Login Module

10.2.2 Bus Search Module

This module enables users to search available buses by selecting:

- Source location
- Destination location
- Travel date

The system retrieves matching buses from the database and displays them to the user.

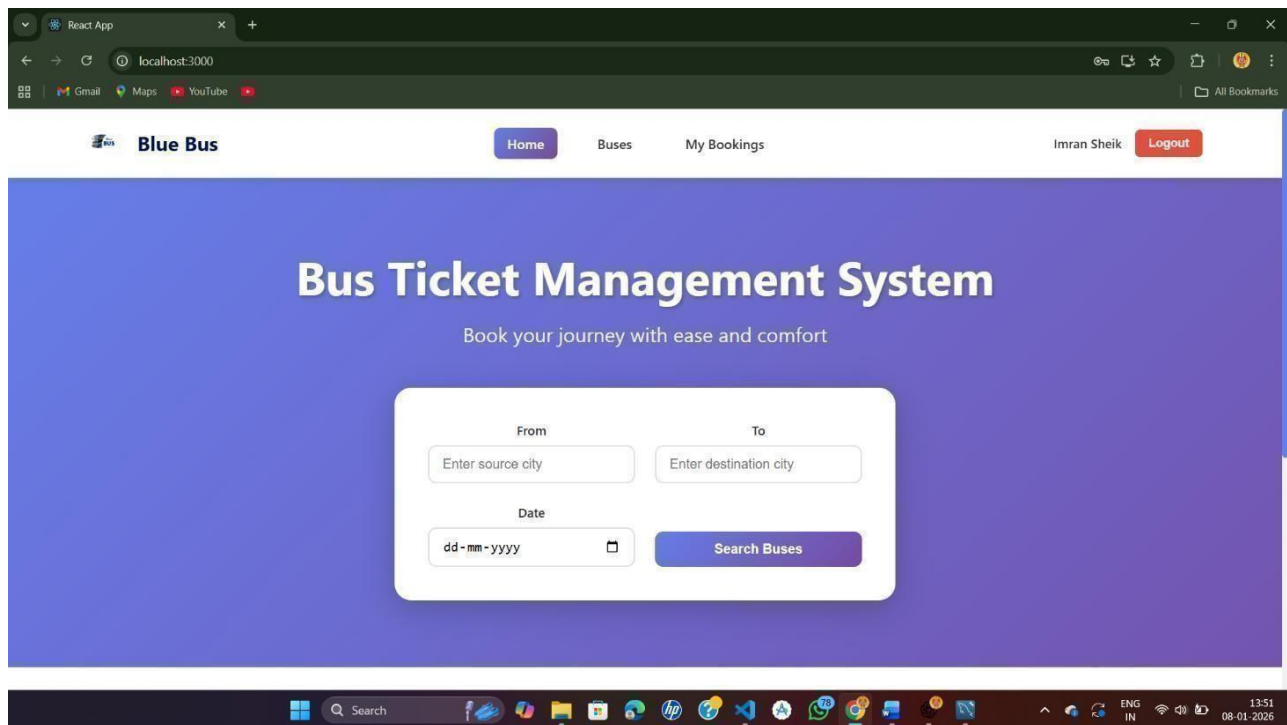


Fig 10.2 Bus Search Module

10.2.3 Ticket Booking Module

This module allows users to:

- Enter passenger details
 - Confirm selected seats
 - Book tickets successfully
- After booking, the ticket details are stored in the database.

Blue Bus Home Buses My Bookings Imran Sheik Logout

Journey Details

Route: amalapuram → razole

Date: Wednesday, December 31, 2025

Time: 10:30 PM

Available Seats: 38

Fare per Seat: ₹600

Passenger Information

Name: Imran Sheik

Email: imransheik@gmail.com

Phone: N/A

Number of Seats *
1
Maximum 38 seats available

Seats: 1 × ₹600

Total Fare: ₹600

Cancel Confirm Booking

Fig: 10.3 Booking Ticket module

10.2.4 Admin Module

The admin can:

- Add new buses
- Update routes and timings
- View bookings
- Manage users

This module helps in effective system administration

Blue Bus Home Buses My Bookings Admin Admin User Logout

Bus Management

+ Add New Bus

Route	Date	Time	Seats	Available	Fare	Actions
amalapuram → razole	Dec 31, 2025	10:30 PM	40	38	₹600	Edit Delete
hyd → mum	Feb 10, 2025	10:30 PM	40	40	₹800	Edit Delete
razole → rajamundry	Dec 28, 2025	11:30 AM	40	40	₹400	Edit Delete
Banglore → Srikakulam	Dec 30, 2025	12:20 PM	30	30	₹500	Edit Delete
amalapuram → razole	Dec 30, 2025	10:30 AM	48	48	₹120	Edit Delete

Fig: 10.4 Admin Login Module

10.3 Running the Code:

10.3.1 Install Java JDK (version 17 or above).

To run the Bus Ticket Reservation System, the following steps are followed:

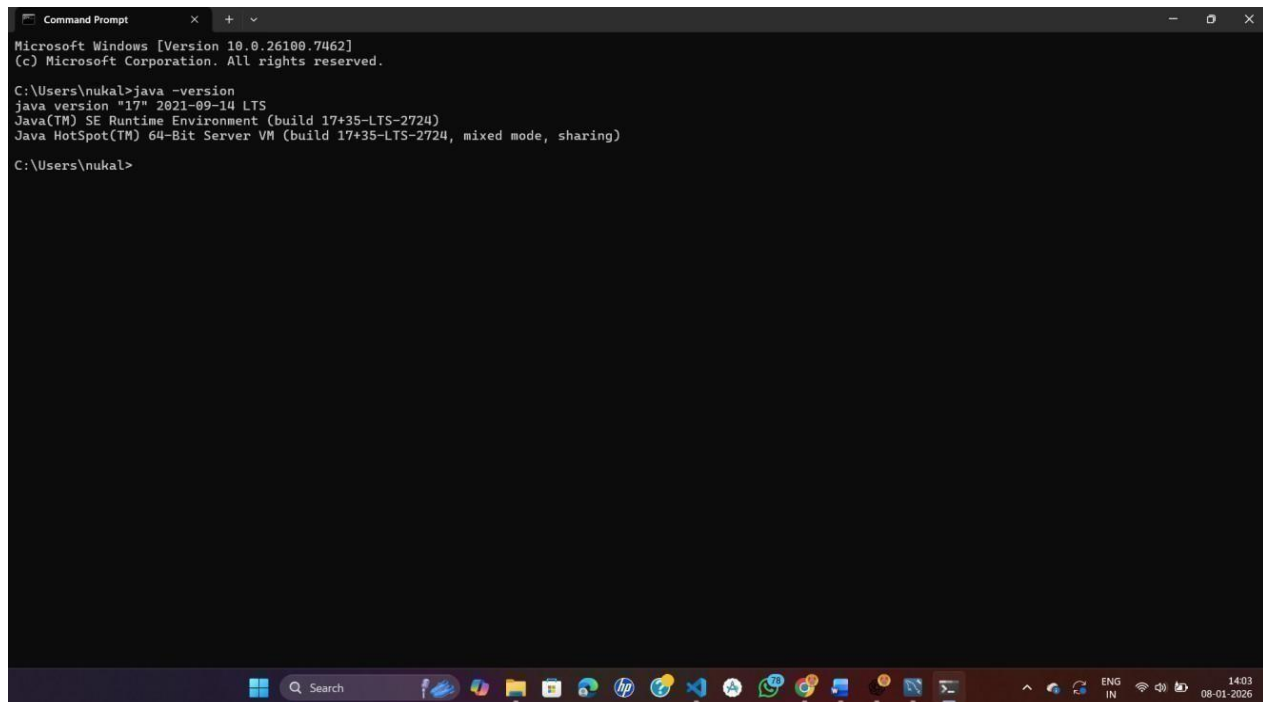


Fig 10.5 Install of Java Development Kit 17

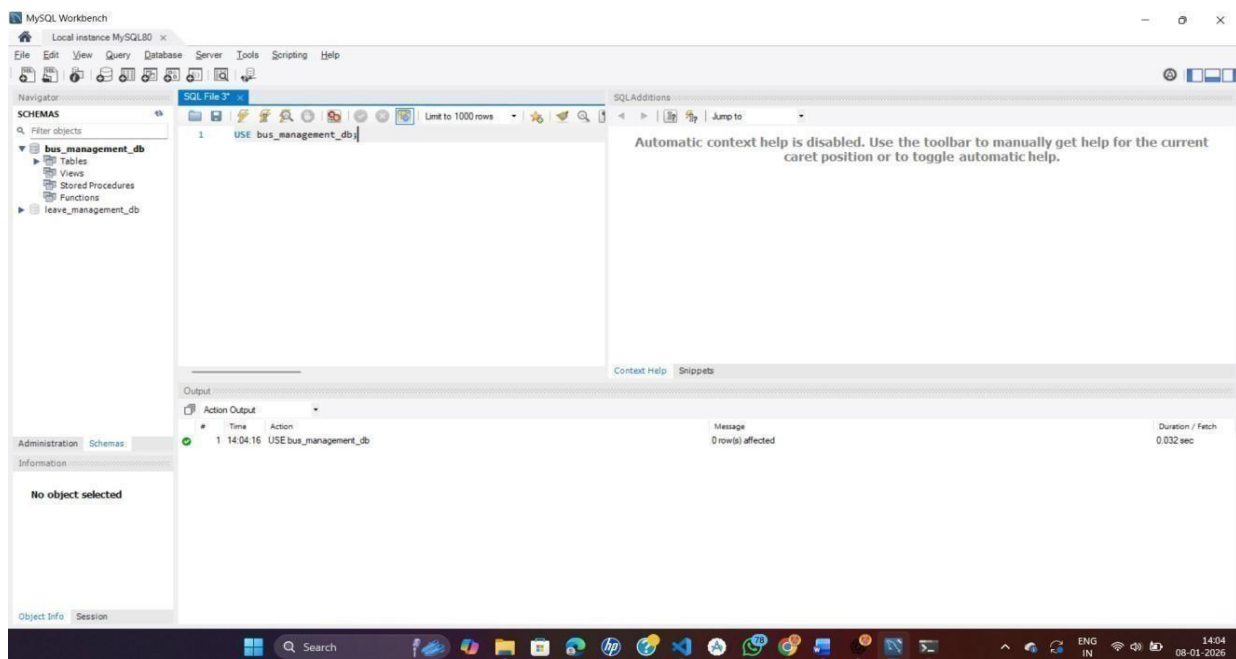


Fig 10.6 Installation of My SQL Server

Open the project in an IDE such as IntelliJ IDEA or VS Code.

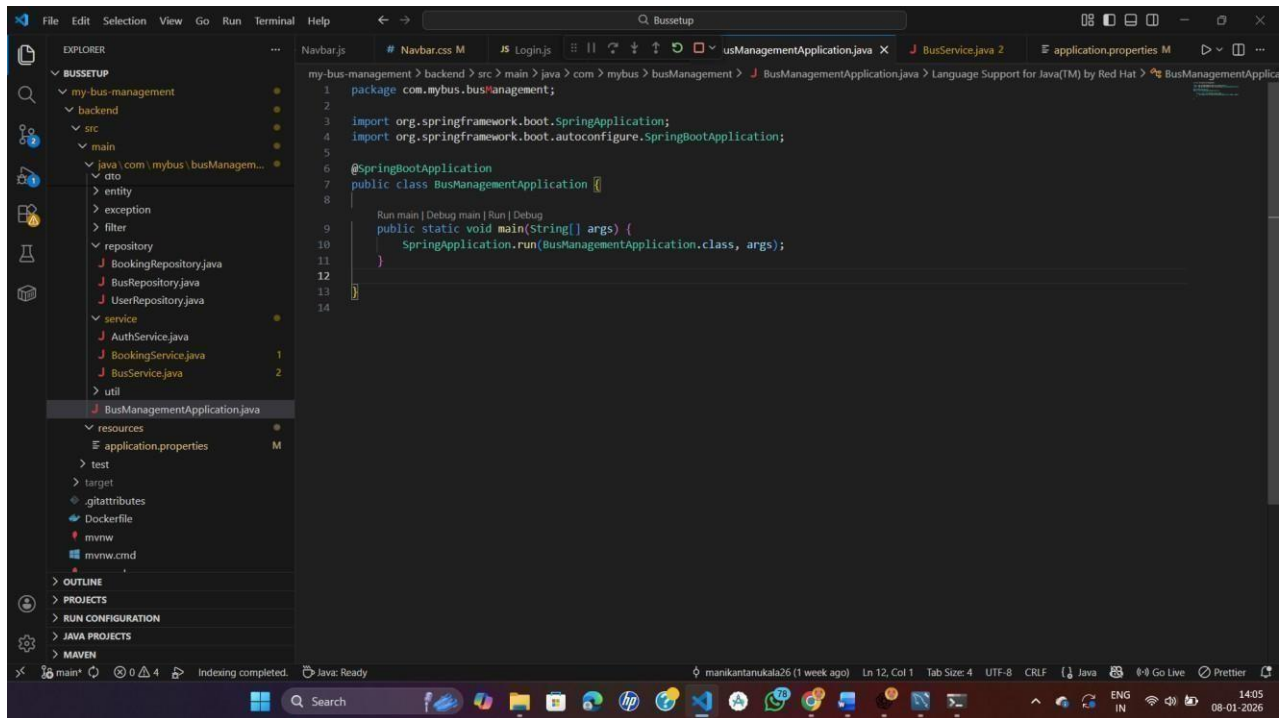


Fig 10.7 Open Project in Visual Studio Code

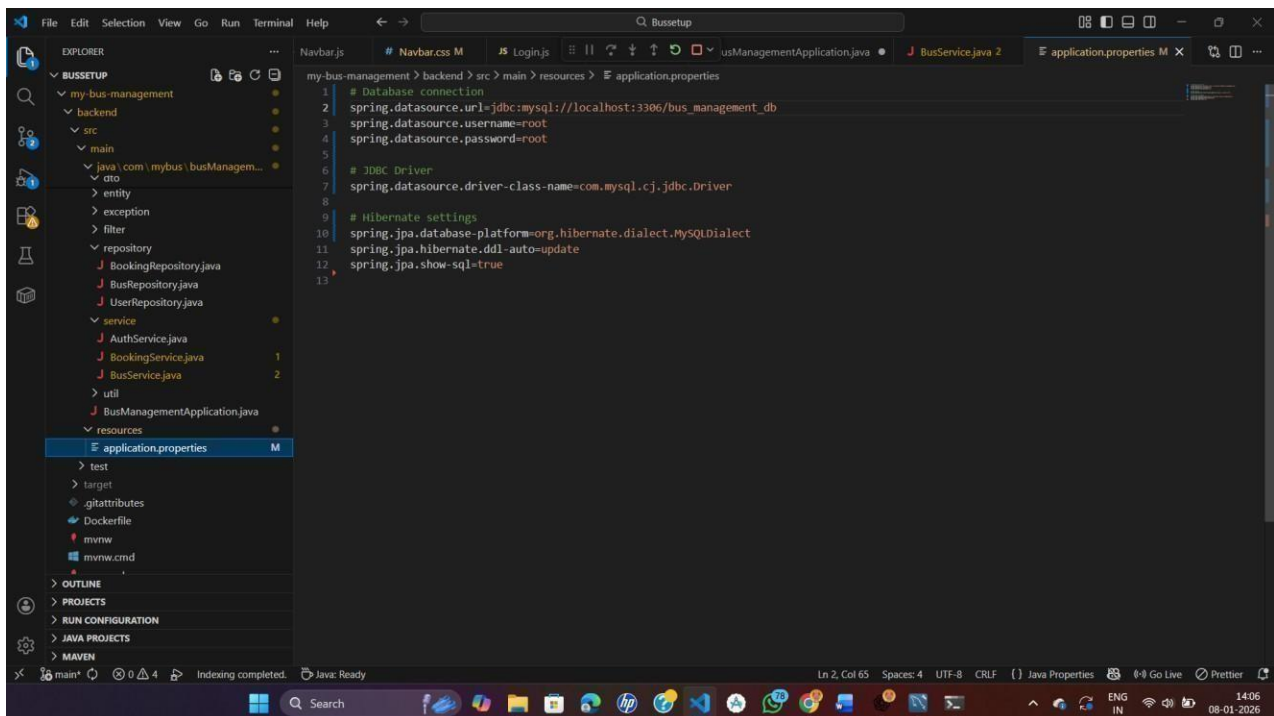


Fig 10.8 Update DataBase Configuration in application.properties

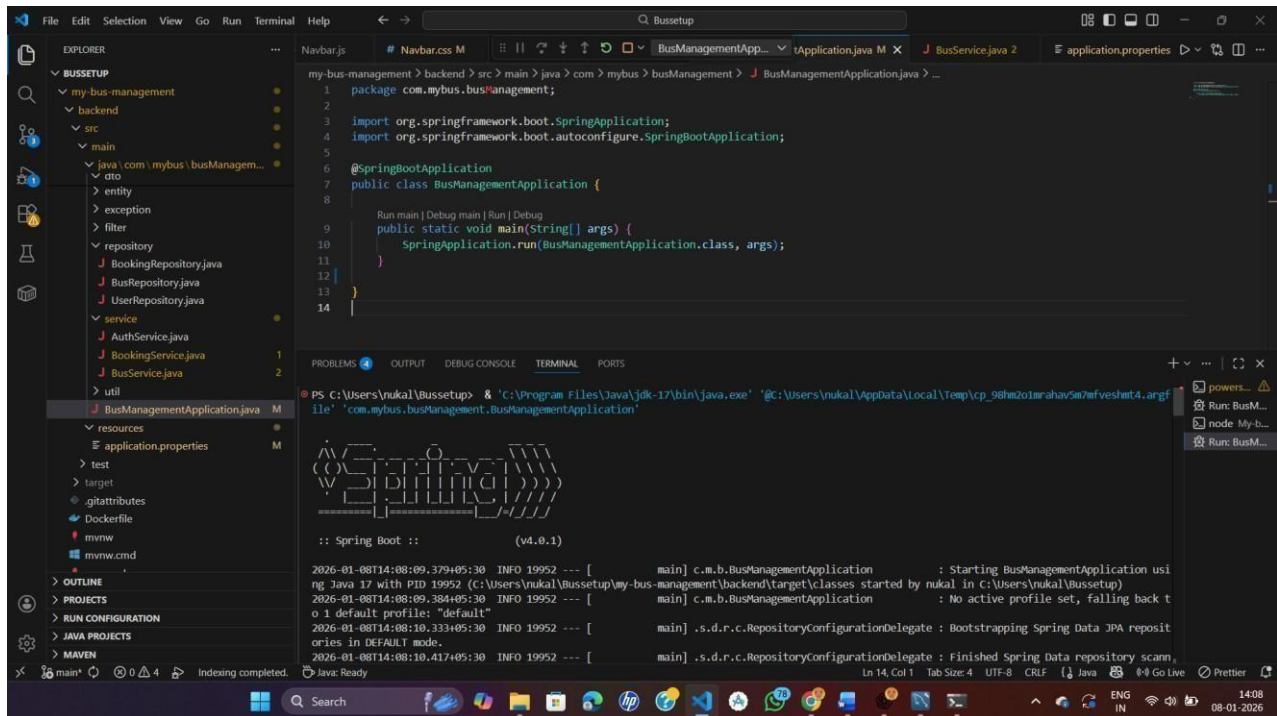


Fig 10.9 Start Application by running Main Application

URL:<http://localhost:8080>

The home page of the Bus Ticket Reservation System is displayed. Users can log in using valid credentials and start booking bus tickets.

This confirms that the application is running successfully and is ready for use.

CHAPTER – 11

CONCLUSION

In conclusion, the Bus Ticket Reservation System has been successfully designed and implemented to automate the traditional bus ticket booking process. The system provides an efficient and user-friendly platform for users to search buses, check seat availability, book tickets, and manage reservations online. By replacing the manual booking process, the system reduces human effort, minimizes errors, and saves time for both passengers and administrators.

The application is developed using Java for backend processing and MySQL for database management, ensuring reliable data storage and secure transaction handling. The admin module allows effective management of buses, routes, schedules, and bookings, while users can easily reserve tickets through a simple interface. Overall, the project meets all its objectives and demonstrates the practical application of software engineering concepts in real-world transportation systems.

Future Work:

In the future, the Bus Ticket Reservation System can be enhanced by integrating online payment facilities such as UPI, debit cards, and credit cards to enable secure cashless transactions. Mobile application support for Android and iOS platforms can be developed to allow users to book tickets conveniently using smartphones. Real-time seat availability and automatic seat locking mechanisms can be implemented to prevent double bookings. The system can also be improved by adding SMS and email notification services for ticket confirmations, cancellations, and travel reminders. Additionally, integrating GPS-based bus tracking will help passengers monitor live bus locations and arrival times. Support for multiple regional languages and advanced reporting features for administrators can further improve usability and efficiency, making the system suitable for large-scale commercial deployment. mapping and automated reporting tools can aid researchers in tracking endangered species and analyzing population trends more effectively.

Final Statement

The Bus Ticket Reservation System is a scalable and efficient solution that simplifies the ticket booking process. With future enhancements, the system can be expanded into a full-fledged commercial application suitable for real-world deployment.

CHAPTER – 12

REFERENCES

1. **Roger S. Pressman**, Software Engineering: A Practitioner’s Approach, McGraw-Hill Education, 8th Edition, 2015.
2. **Herbert Schildt**, **Java: The Complete Reference**, McGraw-Hill Education, 11th Edition, 2019.
3. **Kathy Sierra & Bert Bates**, Head First Java, O’Reilly Media, 2nd Edition, 2018.
4. **Elmasri, R. and Navathe, S. B.**, Fundamentals of Database Systems, Pearson Education, 7th Edition, 2016.
5. **MySQL**
<https://dev.mysql.com/doc/>
(Accessed for database design and SQL queries)
6. **SpringBootOfficial**
<https://spring.io/projects/spring-boot>
(Used for backend application development)
7. **ApacheTomcat**
<https://tomcat.apache.org/>
(Used for deployment of the web application)
8. **Oracle Java**
<https://docs.oracle.com/en/java/>
(Used for Java programming reference)
9. **W3Schools–WebTechnologies**,
<https://www.w3schools.com/>
(Referred for HTML, CSS, JavaScript basics)
10. **GeeksforGeeks–JavaandDBMSTutorials**,
<https://www.geeksforgeeks.org/>
(Used for understanding Java concepts and database connectivity)

Peer Reviewed ISSN Approved | Impact Factor: 8.57

TIJER INTERNATIONAL JOURNAL

International Peer Reviewed & Refereed Journal, Open Access Journal
ISSN 2349-9249 | Impact factor: 8.57 | ESTD Year: 2014

ISSN: 2349-9249

**International Peer Reviewed
Refereed Journals,
Open Access Journal**

TIJER

Scholarly open access journal, Peer-reviewed, and Refereed Journal, Impact factor 8.57 (Calculate by google scholar and Semantic Scholar | AI-Powered Research Tool), Multidisciplinary, Monthly, Indexing in all major database & Metadata, Citation Generator, Digital Object Identifier(DOI)

Website: www.tijer.org

E-mail: editor@tijer.org



Publisher and Managed by: IJ Publication

TIJER INTERNATIONAL JOURNAL

International Peer Reviewed & Refereed Journal, Open Access Journal

ISSN 2349-9249 | Impact factor: 8.57 | ESTD Year: 2014

[ISSN Approved Journal](#)

ISSN: 2349-9249

This work is subjected to be copyright. All rights are reserved whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, re-use of illusions, recitation, broadcasting, reproduction on microfilms or in any other way, and storage in data banks. Duplication of this publication of parts thereof is permitted only under the provision of the copyright law, in its current version, and permission of use must always be obtained from TIJER www.tijer.org Publishers.

TECHNIX INTERNATIONAL JOURNAL FOR ENGINEERING RESEARCH (TIJER) is published under the Name of TIJER publication and URL: www.tijer.org.



©TIJER Journal

Published in India

Typesetting: Camera-ready by author, data conversation by TIJER Publishing Services – TIJER Journal.

TIJER Journal, WWW.TIJER.ORG

ISSN: 2349-9249

TECHNIX INTERNATIONAL JOURNAL FOR ENGINEERING RESEARCH (TIJER) is published in online form over Internet. This journal is published at the Website <http://www.tijer.org>, maintained by TIJER Gujarat, India.

Role-Based Bus Ticket Reservation and Management System Using Spring Boot and RESTful Architecture

M.V.K. Subash¹

¹Project Guide, Department of CSE
Bonam Venkata Chalamayya Engineering College
Affiliated to JNTU Kakinada, Andhra Pradesh-533210, India
mvksubash.bvce@bvcgroup.in

N.V.V.G. Manikanta², K. Hemanth³, K.A.G. Teja⁴

¹²³Department of Computer Science and Engineering
Bonam Venkata Chalamayya Engineering College
Affiliated to JNTU Kakinada, Andhra Pradesh-533210, India
²22221a0582@bvcgroup.in, ³22221a0553@bvcgroup.in,
⁴22221a0551@bvcgroup.in

Abstract—Traditional bus ticket reservation systems often rely on manual counter-based booking processes or loosely integrated digital platforms, resulting in long queuing times, booking conflicts, inefficient seat inventory management, and limited real-time visibility for passengers and operators. This paper presents the design and implementation of a role-based Bus Ticket Reservation and Management System developed using Spring Boot and RESTful architectural principles. The system follows a layered architecture that separates controller, service, repository, and data transfer layers to enhance maintainability and extensibility. Core business rules including seat availability validation, prevention of double booking, role-restricted operations, and controlled booking status transitions are enforced at the service layer with declarative transaction management ensuring data consistency. A key design decision implements atomic seat allocation during booking confirmation, preventing overbooking and resource conflicts. Functional validation confirms the correct enforcement of business rules and transactional behavior.

Index Terms—Bus Ticket Reservation, Spring Boot, RESTful API, Role-Based Access Control, Transaction Management, Seat Allocation, Booking Management System, Layered Architecture, ACID Properties, Database Consistency

I. INTRODUCTION

Public transportation management systems play a vital role in improving passenger experience and operational efficiency in the transit sector. Traditional ticket booking methods such as physical counter reservations or phone-based scheduling often lead to long waiting periods, human errors, booking overlaps, and inefficient utilization of transit resources.

Manual ticketing lacks centralized control and real-time seat availability updates, causing frequent conflicts when multiple passengers attempt to book the same seat. Although digital bus reservation systems have emerged to address these issues, many existing implementations focus primarily on user interfaces while neglecting backend architectural robustness.

This paper presents a Bus Ticket Reservation and Management System developed as an academic prototype to demonstrate modern software engineering practices using Spring Boot and RESTful APIs. The system manages the complete ticket lifecycle from search and booking requests to confirmation and cancellation while enforcing strict business rules and ensuring transactional consistency.

A. Contributions

The key contributions include: (1) layered architectural design using Controller-Service-Repository pattern, (2) comprehensive server-side validation framework, (3) transaction-coordinated booking workflow ensuring ACID properties, (4) RESTful API design with proper endpoint organization, (5) dual-role system architecture with role-based access control, and (6) dynamic seat inventory management with real-time availability tracking.

II. LITERATURE REVIEW

A. Early Web-Based Solutions

Early academic efforts focused on replacing manual ticketing with online forms and database-backed storage. These implementations demonstrated improved processing speed but suffered from procedural designs lacking service-layer abstraction, limiting maintainability and testability.

B. Modern Framework Implementations

Recent studies adopted modern web frameworks, demonstrating responsive frontend design and REST API integration. However, many acknowledged limitations in transaction management, as NoSQL databases' eventual consistency models introduced integrity risks during concurrent booking updates.

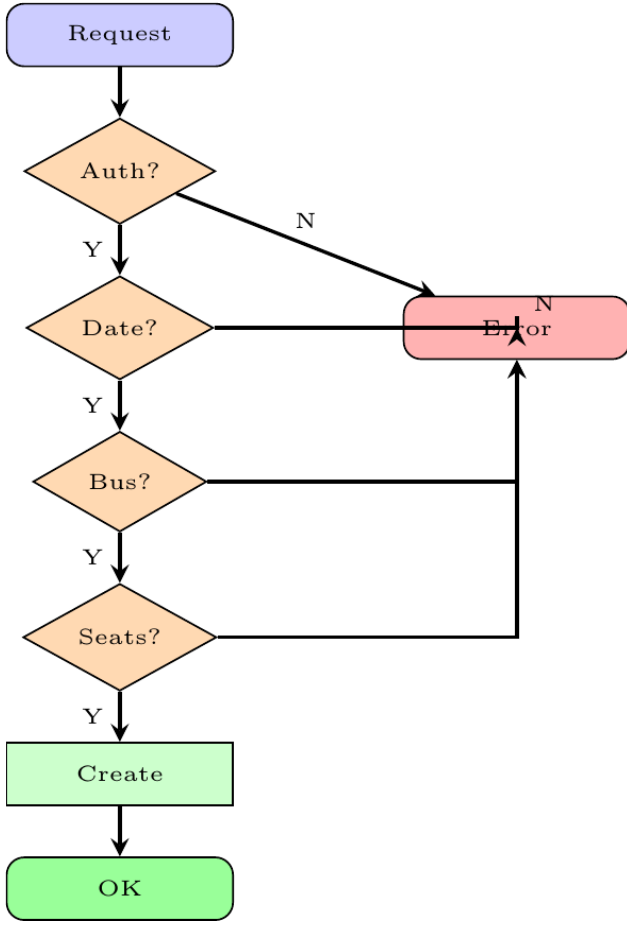


Fig. 1. Booking workflow with validation

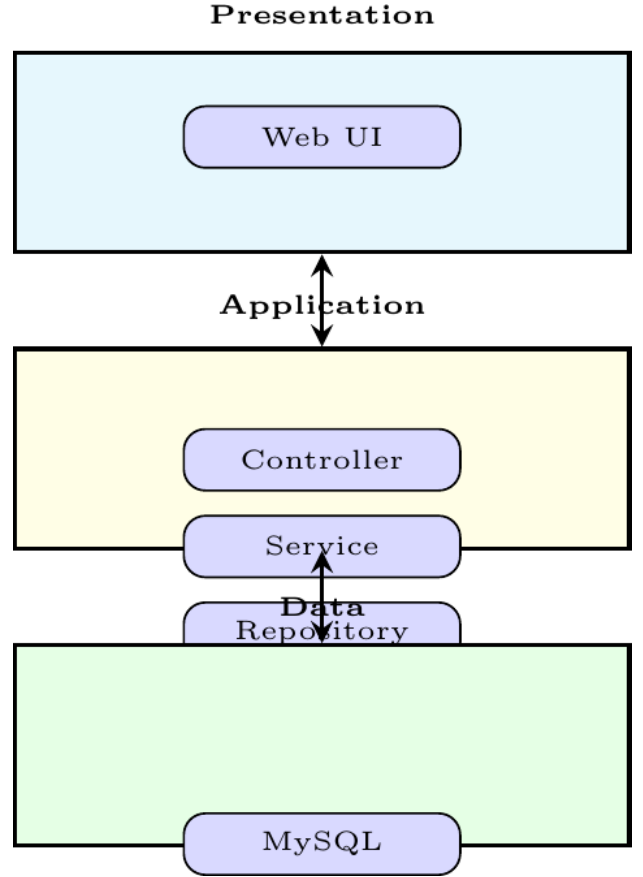


Fig. 2. Three-tier system architecture

C. Identified Gaps

Analysis reveals recurring gaps: (1) lack of clear architectural separation, (2) validation enforced only at UI level, (3) non-atomic booking confirmation workflows, (4) absent overlapping seat allocation detection, (5) limited administrative controls, and (6) inconsistent cancellation handling. Our system addresses these through strict architectural separation, comprehensive server-side validation, and explicit transaction boundaries.

III. SYSTEM ARCHITECTURE

The proposed system adopts a three-tier client-server architecture separating presentation, application logic, and data persistence concerns.

A. RESTful API Design

The application exposes REST endpoints across domain-specific controllers: UserController (registration/authentication), BusController (fleet management/search), BookingController (booking lifecycle), and AdminController (system configuration). Endpoints follow resource-oriented

naming with appropriate HTTP verbs and JSON data interchange format.

B. Data Model and Entity Relationships

The data model comprises four core JPA entities: User (authentication credentials and role designation), Bus (fleet information including capacity and pricing), Booking (reservation details with foreign keys to User and Bus), and Route (route information supporting journey planning). Entity relationships employ JPA annotations with proper cascade configurations and foreign key constraints.

C. Three-Tier Architecture

Presentation Tier: A web-based interface provides bus search, ticket booking, and administrative operations. Communication occurs through RESTful HTTP APIs.

Application Tier: Implemented using Spring Boot 3.x, encapsulating business logic, validation rules, and transaction coordination. Internally follows Model-View-Controller pattern with Controller, Service, and Repository sub-layers.

Data Tier: MySQL relational database managed through Hibernate ORM provides persistent storage with ACID transaction guarantees and referential integrity enforcement.

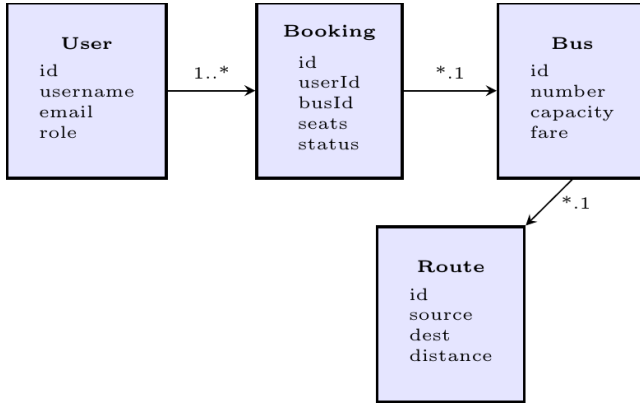


Fig. 3. Core entity relationships

IV. SYSTEM DESIGN AND IMPLEMENTATION

A. Authentication and Role Management

The authentication module implements user registration and login functionality. Registration validates username and email uniqueness before creating User entity with assigned role (USER, ADMIN). The system implements role-based access control at the service layer, ensuring regular users can search buses and manage their bookings, while administrators access extended fleet management and system configuration capabilities.

B. Bus Management Module

Bus registration establishes essential information including identification number, route details, timing information, capacity configuration, and pricing.

Bus Search and Filtering: The system implements advanced search functionality enabling passengers to locate suitable transportation options. Search parameters include source location, destination location, travel date, and seat requirements. The search operation executes database queries identifying buses matching specified criteria with sufficient seat availability.

Administrative Operations: Administrators access extended capabilities including adding new buses, updating information, adjusting seat capacity, modifying fares, controlling operational status, and removing buses from inventory. All modifications execute within transactional boundaries to maintain data consistency.

C. Booking Lifecycle and Validation

Booking Creation Process: Ticket booking submission initiates multi-stage validation: (1) user authentication verification, (2) temporal validation ensuring future travel date, (3) bus existence confirmation, (4) seat availability verification, (5) passenger information validation, (6) double booking prevention, and (7) amount calculation. Upon successful validation, the system creates a Booking entity with CONFIRMED status.

Seat Availability Management: Seat availability tracking implements atomic updates coordinated within database transactions. The booking confirmation process executes operations atomically: retrieve Bus entity, validate requested seats, create Booking record, decrement available seats, and save both entities. The @Transactional annotation establishes transaction boundaries ensuring all operations succeed or fail together.

```

@Transactional
public Booking createBooking(
    BookingRequest request) {
    Bus bus = busRepository
        .findById(request.getBusId())
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Bus not found"));

    if (bus.getAvailableSeats() <
        request.getNumberOfSeats()) {
        throw new InsufficientSeatsException(
            "Only " + bus.getAvailableSeats()
            + " seats available");
    }

    Booking booking = new Booking();
    booking.setBus(bus);
    booking.setNumberOfSeats(
        request.getNumberOfSeats());
    booking.setBookingStatus(
        BookingStatus.CONFIRMED);

    bus.setAvailableSeats(
        bus.getAvailableSeats() -
        request.getNumberOfSeats());

    busRepository.save(bus);
    return bookingRepository.save(booking);
}
  
```

D. Transaction-Coordinated Cancellation Workflow

Booking cancellation demonstrates transaction management principles ensuring atomic seat release and status updates. The cancellation service method executes: retrieve Booking entity, validate CONFIRMED status, retrieve associated Bus entity, update booking status to CANCELLED, restore available seats, and save modified entities. The transactional boundary ensures operations complete atomically.

E. Administrative System Configuration

Route Management: Administrators configure bus routes defining source-destination pairs, distance, journey duration, and operational status.

Dynamic Fare Adjustment: The system supports bulk fare updates, percentage-based adjustments, and absolute fare modifications for individual buses. Fare changes apply immediately to new bookings while preserving historical pricing.

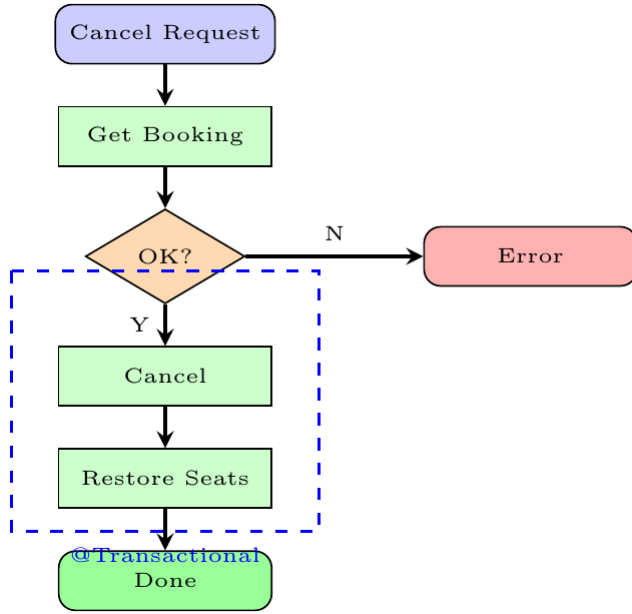


Fig. 4. Cancellation workflow

Seat Capacity Management: Bus capacity modifications enable administrators to increase or decrease total seat count while preventing capacity reductions that would invalidate existing confirmed bookings.

F. Data Transfer Objects and Exception Handling

The implementation employs DTOs separating API contracts from database entities. Request DTOs define client-submitted data with Jakarta Validation annotations. Response DTOs control data exposure, excluding sensitive fields.

Exception handling employs custom exception classes: *InvalidBookingException*, *ResourceNotFoundException*, *InsufficientSeatsException*, *BookingConflictException*, and *UnauthorizedOperationException*. Controllers catch service layer exceptions and construct error response objects with appropriate HTTP status codes, descriptive messages, and timestamps.

G. Comparative Analysis

Compared to conventional manual ticket booking processes, the implemented system provides measurable improvements: reduced booking time (from minutes to seconds), real-time availability visibility, eliminated double booking through automated validation, centralized fleet management reducing operational overhead, comprehensive audit trail enabling accountability, and instant seat release improving inventory utilization.

V. RESULTS AND PERFORMANCE EVALUATION

A. Functional Validation Results

Functional validation was conducted using scenario-based API testing covering typical workflows and error conditions.

Core Validation Rules: Ten core validation rules were verified: (1) date range correctness, (2) seat availability verification, (3) double booking prevention, (4) bus existence

validation, (5) user authentication, (6) status transition enforcement, (7) role-based authorization, (8) booking ownership, (9) passenger information completeness, and (10) amount calculation accuracy. All validation rules demonstrated correct enforcement across standard scenarios and edge cases.

Transaction Management Verification: Transaction management was tested by simulating faults during booking and cancellation workflows. Results confirmed atomicity (complete rollback on failures), consistency (valid database state maintained), isolation (serialized concurrent bookings), and durability (persistence across restarts).

B. System Performance Characteristics

Read Operations: Simple retrieval operations exhibit low latency (100-200ms average) due to indexed database access and minimal business logic processing.

Write Operations: Transactional workflows incur higher overhead (200-300ms average) due to multi-stage validation and coordinated database updates within transactional boundaries.

Scalability Considerations: The system follows Spring Boot's default embedded server configuration, making it suitable for small to medium bus operators under typical usage patterns.

VI. DISCUSSION

A. Design Validation and Key Observations

The implementation successfully demonstrates several software engineering principles. The Controller-Service-Repository pattern provides clear separation of concerns with distinct layer responsibilities, improving code organization and maintainability. Declarative `@Transactional` annotations cleanly handle atomic operations without manual transaction demarcation. The DTO pattern successfully decoupled API contracts from persistence models, enabling independent schema evolution.

B. Implementation Challenges

Several challenges emerged: (1) formulating accurate seat availability queries under concurrent access required careful consideration of transaction isolation levels, (2) implementing comprehensive multi-layer validation without code duplication required thoughtful service layer design, (3) determining appropriate scope for administrative overrides while maintaining system integrity, and (4) ensuring atomic seat release and status updates during cancellations.

C. Limitations and Constraints

Despite functional correctness, the system has limitations: basic authentication without Spring Security integration, stubbed payment processing and refund workflows, absent email/SMS notifications, single-operator deployment only, missing advanced features (seat selection, dynamic pricing, loyalty programs), and unoptimized performance (no caching, pagination, or query optimization).

VII. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper presented a Bus Ticket Reservation and Management System prototype demonstrating modern software engineering practices using Spring Boot and RESTful architecture. The implementation successfully addresses common challenges through layered architectural design, comprehensive server-side validation, transaction-coordinated workflows, role-based access control, RESTful API design, and dynamic administrative capabilities. The atomic seat allocation approach combined with transactional booking workflows prevents overbooking while maintaining system integrity.

B. Future Work

Several enhancements would move this prototype toward production readiness: (1) Spring Security integration with JWT authentication, (2) payment gateway integration supporting multiple payment methods, (3) email/SMS/push notification services, (4) visual seat selection and booking modification capabilities, (5) GPS-based real-time tracking, (6) demand-based dynamic pricing algorithms, (7) comprehensive analytics dashboard, (8) multi-operator platform support, (9) performance optimization with Redis caching and containerization, (10) comprehensive automated testing suite, and (11) native mobile applications. These enhancements would transform the academic prototype into a production-ready platform suitable for real-world deployment.

ACKNOWLEDGMENT

The authors thank Bonam Venkata Chalamayya Engineering College and the Department of Computer Science and Engineering for their support. We express sincere gratitude to our project guide Mr. M.V.K. Subash for his valuable guidance throughout the project.

REFERENCES

- [1] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY: McGraw-Hill Education, 2015.
- [2] H. Schildt, *Java: The Complete Reference*, 11th ed. New York, NY: McGraw-Hill Education, 2019.
- [3] K. Sierra and B. Bates, *Head First Java*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.
- [4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA: Pearson Education, 2016.
- [5] C. Walls, *Spring Boot in Action*. Shelter Island, NY: Manning Publications, 2016.
- [6] S. Kumar and T. R. Singh, "Role Based Access Control for Transportation Management Systems," *International Journal of Computer Science and Information Technologies*, vol. 6, no. 4, pp. 3456–3461, 2015.
- [7] P. Verma, R. Sharma, and A. Kumar, "A Survey on Online Bus Ticket Reservation Systems," *International Journal of Advanced Research in Computer Science*, vol. 8, no. 7, pp. 2340–2345, 2017.
- [8] M. Patel and N. Shah, "Design and Implementation of Web-Based Bus Reservation System Using Spring Framework," *International Journal of Engineering Research and Technology*, vol. 7, no. 3, pp. 215–220, 2018.
- [9] A. Singh, K. Mishra, and R. Gupta, "Transaction Management in Online Booking Systems: Challenges and Solutions," *Journal of Database Management*, vol. 30, no. 2, pp. 45–62, 2019.
- [10] MySQL Documentation. [Online]. Available: <https://dev.mysql.com/doc/>
- [11] Spring Boot Official Documentation. [Online]. Available: <https://spring.io/projects/spring-boot>

- [12] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

TIJER INTERNATIONAL JOURNAL

International Peer Reviewed & Refereed Journal, Open Access Journal

ISSN 2349-9249 | Impact factor: 8.57 | ESTD Year: 2014

Scholarly open access journal, Peer-reviewed, and Refereed Journal, Impact factor 8.57 (Calculate by google scholar and Semantic Scholar | AI-Powered Research Tool), Multidisciplinary, Monthly, Indexing in all major database & Metadata, Citation Generator, Digital Object Identifier(DOI)

Impact Factor : 8.57 (calculated by google scholar)	Monthly, Multidisciplinary and Multilanguage (Regional language supported)
International Journal	Peer Review Journal Refereed Journal
Journal Soft copy, Research Paper, Certificate, DOi and Hard copy of Journal Provided.	Indexing In Google Scholar, SSRN, ResearcherID-Publons, Semantic Scholar AI-Powered Research Tool, Microsoft Academic, Academia.edu, arXiv.org, Research Gate, CiteSeerX, ResearcherID Thomson Reuters, Mendeley : reference manager, DocStoc, ISSUU, Scribd, and many more
Automated Metadata Citation Generator	Fast Process and Low Publication Charge
Provide the Mail and SMS notification	Approved, Open Access Journal
Highly Secure SSL Website	Managed By IJPUBLICATION

Submit Your Manuscript/Papers To
editor@tijer.org | www.tijer.org



©TIJER , All Rights Reserved | www.tijer.org | editor@tijer.org



TIJER - INTERNATIONAL RESEARCH JOURNAL

TIJER.ORG | ISSN : 2349-9249

An International Open Access, Peer-reviewed, Refereed Journal

The Board of
TIJER - INTERNATIONAL RESEARCH JOURNAL

Is hereby awarding this certificate to

Nukala V V G Manikanta

In recognition of the publication of the paper entitled

**Role-Based Bus Ticket Reservation and Management System Using Spring
Boot and RESTful Architecture**

Published in Volume 13 Issue 2, February-2026, | Impact Factor: 8.57 by Google Scholar

Co-Authors - K Hemanth, K A G Teja, M V K Subash

Paper ID - TIJER2602031



Registration ID - 161099

An International Scholarly Open Access Journals, Peer-Reviewed, & Refereed Journals, AI-Powered Research Tool, Multidisciplinary, Monthly, Online, Print Journal, Indexed Journal

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

TIJER - INTERNATIONAL RESEARCH JOURNAL

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.tijer.org | Email: editor@tijer.org | ESTD: 2014

Manage By: IJPUBLICATION Website: www.tijer.org | Email ID: editor@tijer.org

Certificate of Publication

TIJER | ISSN : 2349-9249



TIJER - INTERNATIONAL RESEARCH JOURNAL

TIJER.ORG | ISSN : 2349-9249

An International Open Access, Peer-reviewed, Refereed Journal

The Board of
TIJER - INTERNATIONAL RESEARCH JOURNAL

Is hereby awarding this certificate to

K Hemanth

In recognition of the publication of the paper entitled

**Role-Based Bus Ticket Reservation and Management System Using Spring
Boot and RESTful Architecture**

Published in Volume 13 Issue 2, February-2026, | Impact Factor: 8.57 by Google Scholar

Co-Authors - Nukala V V G Manikanta, K A G Teja, M V K Subash

Paper ID - TIJER2602031



Registration ID - 161099

An International Scholarly Open Access Journals, Peer-Reviewed, & Refereed Journals, AI-Powered Research Tool, Multidisciplinary, Monthly, Online, Print Journal, Indexed Journal

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

TIJER - INTERNATIONAL RESEARCH JOURNAL

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.tijer.org | Email: editor@tijer.org | ESTD: 2014

Manage By: IJPUBLICATION Website: www.tijer.org | Email ID: editor@tijer.org

Certificate of Publication

TIJER | ISSN : 2349-9249



TIJER - INTERNATIONAL RESEARCH JOURNAL

TIJER.ORG | ISSN : 2349-9249

An International Open Access, Peer-reviewed, Refereed Journal

The Board of
TIJER - INTERNATIONAL RESEARCH JOURNAL

Is hereby awarding this certificate to

K A G Teja

In recognition of the publication of the paper entitled

**Role-Based Bus Ticket Reservation and Management System Using Spring
Boot and RESTful Architecture**

Published in Volume 13 Issue 2, February-2026, | Impact Factor: 8.57 by Google Scholar

Co-Authors - Nukala V V G Manikanta, K Hemanth, M V K Subash

Paper ID - TIJER2602031



Registration ID - 161099

An International Scholarly Open Access Journals, Peer-Reviewed, & Refereed Journals, AI-Powered Research Tool, Multidisciplinary, Monthly, Online, Print Journal, Indexed Journal

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

TIJER - INTERNATIONAL RESEARCH JOURNAL

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.tijer.org | Email: editor@tijer.org | ESTD: 2014

Manage By: IJPUBLICATION Website: www.tijer.org | Email ID: editor@tijer.org

Certificate of Publication

TIJER | ISSN : 2349-9249



TIJER - INTERNATIONAL RESEARCH JOURNAL

TIJER.ORG | ISSN : 2349-9249

An International Open Access, Peer-reviewed, Refereed Journal

The Board of
TIJER - INTERNATIONAL RESEARCH JOURNAL

Is hereby awarding this certificate to

M V K Subash

In recognition of the publication of the paper entitled

**Role-Based Bus Ticket Reservation and Management System Using Spring
Boot and RESTful Architecture**

Published in Volume 13 Issue 2, February-2026, | Impact Factor: 8.57 by Google Scholar

Co-Authors - Nukala V V G Manikanta, K Hemanth, K A G Teja

Paper ID - TIJER2602031



Registration ID - 161099

An International Scholarly Open Access Journals, Peer-Reviewed, & Refereed Journals, AI-Powered Research Tool, Multidisciplinary, Monthly, Online, Print Journal, Indexed Journal

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

TIJER - INTERNATIONAL RESEARCH JOURNAL

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.tijer.org | Email: editor@tijer.org | ESTD: 2014

Manage By: IJPUBLICATION Website: www.tijer.org | Email ID: editor@tijer.org

Certificate of Publication

TIJER | ISSN : 2349-9249